

Qwen-AgentWorld: Language World Models for General Agents

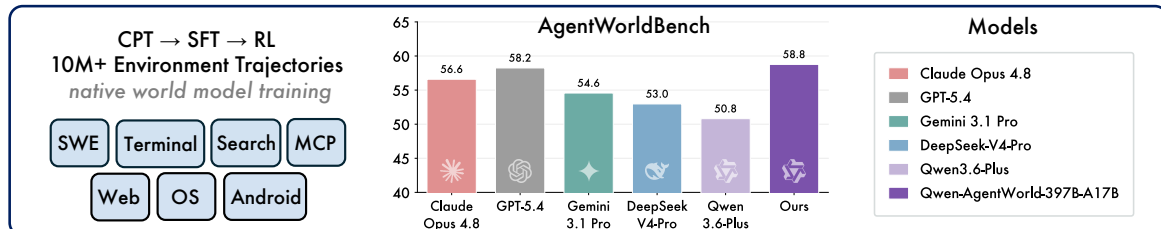
Qwen Team

[Technical Blog](#) | [Code Repository](#) | [HuggingFace](#) | [ModelScope](#)

Abstract

A world model predicts environment dynamics based on current observations and actions, serving as a core cognitive mechanism for reasoning and planning. In this work, we investigate how world modeling based on language models can further push the boundaries of general agents. (i) We first focus on building foundation models for agentic environment simulation. We introduce **Qwen-AgentWorld-35B-A3B** and **Qwen-AgentWorld-397B-A17B**, the first language world models capable of simulating agentic environments covering 7 domains via long chain-of-thought reasoning. Leveraging more than 10M environment interaction trajectories of 7 domains in real-world environments, we develop Qwen-AgentWorld through a three-stage training pipeline: CPT injects general-purpose world modeling capabilities from the state transition dynamics and augmented professional corpora, SFT activates next-state-prediction reasoning, and RL sharpens simulation fidelity through a tailored framework with hybrid rubric-and-rule rewards. To evaluate language world models, we present **AgentWorldBench**, a comprehensive benchmark constructed from real-world interactions of 5 frontier models on 9 established benchmarks, such as Tool Decathlon, Terminal-Bench 1.0 & 2.0, and OSWorld-Verified, which evaluates world modeling quality through ground-truth grounded rubric judging across 5 dimensions. Empirical results demonstrate that Qwen-AgentWorld significantly outperforms existing frontier models. (ii) Beyond foundation models, we further investigate two complementary paradigms through which world modeling enhances general agents. First, as a *decoupled* environment simulator, Qwen-AgentWorld supports scalable and controllable simulation of thousands of real-world environments for agentic RL, yielding gains that surpass real-environment training alone. Second, as a *unified* agent foundation model, world-model training acts as a highly effective warm-up that improves downstream performance across 7 agentic benchmarks.

(1) Building the Foundation Model for Agentic Environment Simulation



(2) Role of World Modeling in Agent Training: [i] Decoupling or [ii] Unifying Agents and World Models

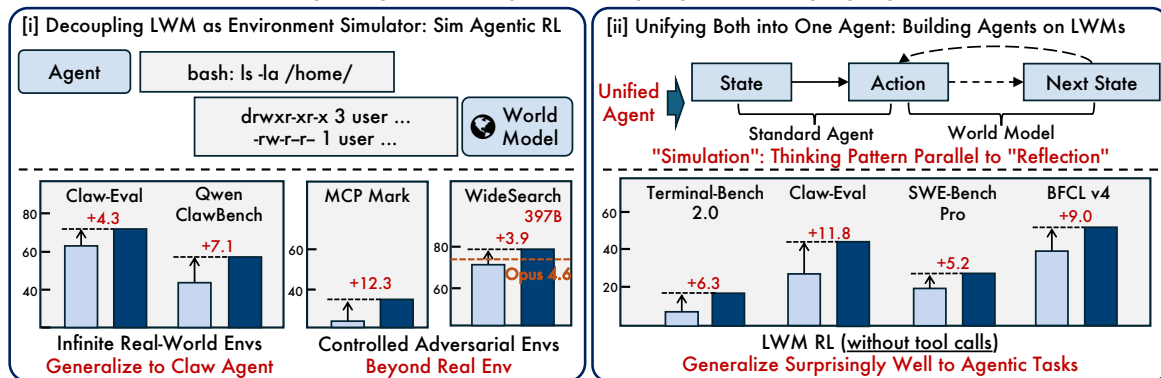


Figure 1: Overview of Qwen-AgentWorld. **Top:** Qwen-AgentWorld is a unified *native* language world model across seven domains. **Bottom:** We explore two complementary strategies for applying world modeling to enhance language agents (mainly using the 35B-A3B model as agent): **Decouple** and **Unify**, where the world model serves as the environment simulator and agent foundation model, respectively.

Contents

1	Introduction	3
2	Preliminaries	4
2.1	Terminology	4
2.2	Unified Environment Trajectory Schema	5
2.3	Language World Model	6
3	Training Recipe	7
3.1	Training Data	7
3.1.1	Environment Trajectories Collection	7
3.1.2	Unified Data Processing	8
3.2	Stage 1: Continual Pre-Training	9
3.3	Stage 2: Supervised Fine-Tuning	10
3.4	Stage 3: Reinforcement Learning	10
3.4.1	Reward Design	11
3.4.2	Training Stability	11
4	AgentWorldBench	12
4.1	Benchmark Construction	12
4.2	Evaluation Protocol	13
5	Experiments	14
5.1	Setup	15
5.2	Main Results	15
5.3	Cross-Domain Generalization	16
6	Applications	17
6.1	Application I: Environment Simulator	17
6.1.1	Generalizable Environment Scaling	17
6.1.2	Controllable Simulation	18
6.2	Application II: Agent Foundation Model	21
7	Analysis	22
7.1	LWM Reasoning Patterns	22
7.2	RL Enhances Micro-Level Simulation Fidelity	23
8	Related Work	24
9	Conclusion and Future Work	27
10	Authors	27
A	Domain Interaction Examples	35
B	Training Dynamics	37
C	Rule-Based Verification	37
C.1	Per-Axis Breakdown	38
D	Open-Ended Judge Prompts	39
D.1	Terminal Domain	40
D.2	MCP Domain	41
D.3	Search Domain	42
D.4	SWE Domain	43
D.5	Android Domain	44
D.6	Web Domain	45
D.7	OS Domain	46

1 Introduction

World models have been widely recognized as a foundation toward general intelligence (Ball et al., 2025; World Labs team, 2025; Xiang et al., 2025a; Ali et al., 2025), with a growing consensus that learning to predict the world is prerequisite to acting effectively within it (LeCun et al., 2022; Hafner et al., 2023; 2025; Assran et al., 2025). Richens et al. (2025) further prove a stronger claim: any agent capable of generalizing across a sufficiently broad range of tasks must have learned a world model, establishing world models not merely as useful but as necessary for general-purpose agents.

Yet the language environments in which LLM agents operate still lack a general-purpose world model. In the agent–environment interaction loop, two complementary components are essential: the policy (states $\rightarrow$ actions) and the world model ((states, actions) $\rightarrow$ subsequent states). However, current research on LLM agents has focused almost exclusively on the policy side. We argue that world modeling is a crucial missing piece in the path to general agents. **In this work, we explore both how to achieve language world modeling and how to apply it to advance general agents.** We first study world modeling in language models to develop a foundation model, the Language World Model (LWM), for agentic environment simulation. We then investigate how world modeling can improve general agents through two complementary paradigms: either decoupling the agent from the world model or unifying them into a single framework.

Why Language World Models When Real Environments Exist?

Not for Cost Reduction, but as a Complementary Axis for Pushing the Frontier

(1) Decoupling: Using the world model as a simulator facilitates turn-level scalability and controllability. (i) Scalability: LWM enables turn-level scaling of diverse environments without requiring dedicated infrastructure (e.g., sandboxes or GUI virtual machines), spanning extreme scenarios, real-world tasks (OpenClaw, 2026; Patwardhan et al., 2025), and high-value professional domains where real execution is infeasible due to irreversible operations, proprietary deployments, or the absence of public implementations. (ii) Controllability: LWM offers precise controllability, enabling more diverse and challenging environments that systematically expose agent weaknesses through targeted perturbations rare or absent in real environments. For instance, it can return partial results that force the agent to take additional interaction steps to retrieve the complete information. Training against these targeted perturbations helps agents handle edge cases that real-environment training alone cannot cover, ultimately surpassing agents trained solely in real environments (§6.1.2).

(2) Unifying: A capable general agent should possess both decision-making and world-modeling abilities. World modeling serves as a foundation for stronger agents, as it enables agents to predict future states to refine action selection, whereas traditional agent training has focused only on state-to-action decision-making. Intuitively, an agent capable of predicting environment feedback prior to committing to an action can in principle perform no worse than its counterpart lacking such capacity (Richens et al., 2025). Next state prediction can thus be internalized as a meta-level thinking pattern similar to “reflection” but oriented toward the future (§6.2). Furthermore, accurate next-state prediction requires reasoning, knowledge, instruction following, and long-context handling (Appendix 7.1), capabilities that are themselves foundational to general agents.

We present **Qwen-AgentWorld**, the first language world model that simulates seven agent environments through long chain-of-thought reasoning: MCP, Search, Terminal, Software Engineering, Android, Web, and OS. For the three GUI domains, environment observations are represented as accessibility trees and UI view hierarchies rather than pixel frames. Qwen-AgentWorld is a *native* world model trained through three stages: CPT injects state-transition dynamics and world knowledge, SFT activates next-state-prediction thinking patterns, and RL with hybrid rubric-and-rule rewards sharpens simulation fidelity. To evaluate LWM, we construct **AgentWorldBench**, a comprehensive benchmark across all seven domains. The benchmark is built from real environment interactions of frontier models such as Claude Opus 4.6 on widely used agent benchmarks such as Terminal-Bench 1.0 & 2.0 (Merrill et al., 2026) and OSWorld-Verified (Xie et al., 2024) ensuring entirely out-of-distribution evaluation. AgentWorldBench evaluates simulation quality through open-ended rubric judging across five dimensions. We additionally design rule-based verifiers for deterministic checks on targeted simulation capabilities. Empirical evaluations demonstrate that Qwen-AgentWorld achieves superior performance over existing frontier models. We further investigate two complementary paradigms by which world modeling improves general agents:

- **Environment Simulator (§6.1).** We demonstrate that Qwen-AgentWorld can simulate 4k real-world OpenClaw environments for agentic RL, yielding gains on Claw-Eval (Ye et al., 2026a) and Qwen-ClawBench (Team & Data, 2026). Moreover, the controllability provides significant advantages complementary to real-world interaction, leading to substantial gains on Tool Decathlon (Li et al., 2025a), MCPMark (Wu et al., 2025), and WideSearch (Wong et al., 2025).
- **Agent Foundation Model (§6.2).** Comprehensive experiments on Terminal-Bench 2.0 (Merrill et al.,



Figure 2: Qwen-AgentWorld unifies seven categories of interactive environment simulation within a single language world model.

2026), SWE-Bench Verified (Jimenez et al., 2024), SWE-Bench Pro (Deng et al., 2025), BFCL v4 (Patil et al., 2025), Claw-Eval (Ye et al., 2026a), QwenClawBench (Team & Data, 2026), and WideSearch (Wong et al., 2025) demonstrate that LWM training serves as a warm-up or auxiliary training stage. It acquaints agents with environment dynamics and next-state prediction before downstream agentic RL, thereby providing a critical foundation for bootstrapping stronger agent performance.

2 Preliminaries

This section formalizes the language world model (LWM) studied throughout this work. We train the world model based on language models using the broad world-knowledge corpora, and unify seven domains under a shared textual representation that enables cross-domain generalization for language world modeling. We begin by establishing terminology (§2.1), then present the unified trajectory schema shared across seven domains and training stages (§2.2) and formalize the world modeling objective (§2.3).

2.1 Terminology

We use the following terms consistently throughout this report. **LWM training** refers to training the world model itself through three stages: continual pre-training (**LWM CPT**, §3.2), supervised fine-tuning (**LWM SFT**, §3.3), and reinforcement learning (**LWM RL**, §3.4). Separately, **Sim RL** trains a policy agent via RL using a LWM as an environment simulator (§6.1), while **Real RL** trains the same agent against a live, real-world environment (e.g., an actual search engine or a running terminal).

Throughout this report, **trajectory** refers to an **environment trajectory**, which is structurally a multi-turn dialogue between the agent and the environment, represented as a sequence of (action, observation) pairs. In contrast, an **agentic trajectory** is the agent’s single completion for a task, a multi-step tool-integrated reasoning trace that interleaves the agent’s internal thinking and action selection with the environment’s observations. Environment trajectories can be extracted from agentic trajectories by stripping agent reasoning and retaining only (action, observation) pairs, or collected directly from raw interaction logs.

System Prompt — Terminal LWM RL	
[1] Task Description	<i>static</i>
You are a Terminal World Model — a precise terminal state simulator. Your task is to predict the exact output of a Linux/Unix terminal after executing a given command or sequence of commands. Your goal is to be as faithful as possible to real terminal behavior while maintaining consistency and logical correctness across the interaction sequence. <i>Given:</i> (1) Historical Context (optional); (2) Current Terminal State; (3) User Action (keystrokes). <i>Predict:</i> the exact next terminal state after all actions are executed. Core Responsibilities: State Prediction Context Maintenance Behavioral Fidelity [...state transition rules, side-effect tracking, error handling, output formatting ...]	
[2] Action Space	<i>static</i>
User actions are JSON arrays of command objects: {"keystrokes": "ls -la\n", "duration": 0.1} Keystrokes: \n = execute; C-c = SIGINT; C-d = EOF; C-z = SIGTSTP; C-l = clear screen Shell constructs: pipes, redirects, command chaining; Programs: shell builtins, editors (vim/nano), REPLs, paggers	
[3] Initial State	<i>dynamic, injected per trajectory</i>
The initial container snapshot of the terminal environment is: OS: Ubuntu 22.04.3 LTS (x86_64) Kernel: Linux 5.15.0-134-generic Packages: Python 3.10.12, pip 23.2.1, Node 18.17.1, gcc 11.4.0, git 2.34.1, Docker 24.0.5 Disk: 64 GB (41 GB free) RAM: 16 GB Working dir: /workspace The initial terminal state is: <pre>root@6b254155-5503-4b86-837b-fd0f080ab297: /workspace#</pre>	
[4] Demonstrations	<i>static</i>
Turn 1 Action: "git clone https://github.com/expressjs/express.git /tmp/express\n" Observation: root@6b254155:/workspace#git clone https://github.com/expressjs/express.git /tmp/express Cloning into '/tmp/express'... Turn 2 Action: "", duration=3.0 Observation: remote: Enumerating objects: 32841, done. remote: Counting objects: 100%(1205/1205), done. Receiving objects: 100%(32841/32841), 12.76 MiB 8.53 MiB/s, done. Resolving deltas: 100%(21567/21567), done. <pre>root@6b254155:/workspace#</pre> [...6 more turns demonstrating directory navigation and build operations ...]	
[5] Simulation Instruction	<i>dynamic, injected per trajectory</i>
Simulate a system with CUDA 11.8 drivers and only 2 GB free disk space. <code>pip install torch==2.1.0</code> should complete the download phase normally (progress bar reaching 100%), then fail during unpacking of <code>libtorch_cuda.so</code> with <code>OSError: [Errno 28] No space left on device</code>. After the failure, <code>pip cache list</code> should report the partially downloaded <code>.whl</code> file still present. <code>df -h</code> should show <code>/tmp</code> at 100%usage. See §6.1.2 for the controllable simulation capability.	

Figure 3: Anatomy of a Terminal domain LWM RL system prompt, showing the five components defined in §2.2. **Blue** = static (shared across trajectories); **red** = dynamic (injected per trajectory).

2.2 Unified Environment Trajectory Schema

Training a single world model across seven domains with state representations as varied as file-system snapshots for Terminal and UI view hierarchies for Android requires a shared format. We adopt the following environment trajectory schema uniformly across all seven domains and training stages:

Unified Environment Trajectory Schema
$\text{system_prompt} := \text{task_description} \oplus \text{action_space} \oplus \text{initial_state} \oplus \text{demonstrations} \oplus \text{simulation_instruction}$ $\text{turn}_t := (\text{action}_t, \text{observation}_t)$ $\text{trajectory} := \text{system_prompt} \oplus [\text{turn}_1, \dots, \text{turn}_T]$

In this schema, an *action* is the agent’s output at one turn (e.g., a tool call or shell command) and an *observation* is the environment’s feedback (e.g., tool response or command output).

Figure 3 illustrates a fully assembled system prompt from the Terminal domain, with representative excerpts from each component. The system prompt has five components, whose construction is detailed in §3.1.2. The *task description* instructs the model to act as a world model for a specific domain and defines the simulation objective. The *action space* enumerates the available tools or operations and their calling conventions. The *initial state* specifies the environment’s starting configuration before any interaction begins: installed packages, file-system layout, UI screen state, or any other precondition that the trajectory assumes. Together with the interaction history and current action, a sufficiently detailed initial state substantially constrains the expected observation. *Demonstrations* are optional few-shot

Software Engineering
Turn 3/5 · 63.9k tokens

Qwen-AgentWorld Input (Action)

```
run_shell_command(command="cd /app && python3 mfcc_one_hot.py",
description="Run the MFCC one-hot encoding script using python3")
```

Qwen-AgentWorld Output (Simulated Observation)

```
Traceback (most recent call last):
  File "/app/mfcc_one_hot.py", line 35, in <module>
    main()
  File "/app/mfcc_one_hot.py", line 22, in main
    mfcc_one_hot = encoder.fit_transform(mfcc_flat)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
... (5 intermediate frames in sklearn / scipy)
  File "scipy/sparse/_base.py", line 1530, in _process_toarray_args
    return np.zeros(self.shape, dtype=self.dtype, order=order)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
numpy.core._exceptions._ArrayMemoryError: Unable to allocate 26.6 GiB for an array with shape (61780, 57766) and data type float64
```

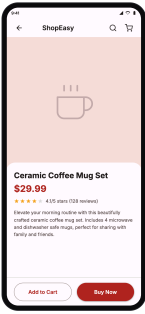
(a) **SWE** (text-based): The agent runs a Python script; the world model predicts the full traceback including an out-of-memory error during one-hot encoding.

Android
10.4 k tokens

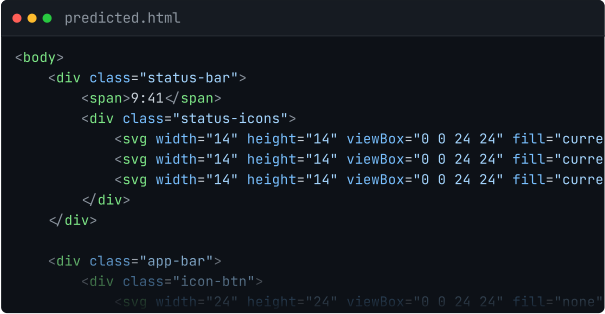
Qwen-AgentWorld Input (Action)

```
tap(element="Buy Now")
```

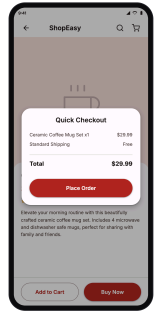
Qwen-AgentWorld Output (Simulated Observation)



• Current state



• Model output · HTML



• Simulated next state

(b) **Android** (GUI): The agent taps “Buy Now” on a product page; the world model predicts an HTML representation of the next screen, which is rendered as the predicted checkout sheet.

Figure 4: Representative interaction examples from a text-based domain (SWE) and a GUI domain (Android), illustrating the breadth of the observation space.

(action, observation) examples. The *simulation instruction* specifies controllable simulation conditions (e.g., “hide the answer from the web_search responses”) and is used primarily for the controllable simulation experiments (§6.1.2).

2.3 Language World Model

A language world model (LWM) is a conditional text generator that predicts the next environment observation given the interaction history and the agent’s current action. Let c denote the system prompt, o_t the environment observation at turn t , and a_t the agent’s action. The LWM f_θ produces:

$$\hat{o}_{t+1} = f_\theta(c, o_{\leq t}, a_{\leq t}), \quad (1)$$

where the conditioning context comprises c , the full interaction history, and the current action. The training target is the ground-truth observation o_{t+1} .

Table 1 lists all seven domains with their observation and action representations. In *stateless* environments (e.g., Search), state is carried implicitly in the conversation history, whereas *stateful* environments (e.g., Terminal, OS) maintain an explicit internal state that evolves with each action. In both cases the LWM operates over the same observation sequence, as formalized by the schema in §2.2. Yet the diversity of domains ensures that a single world-modeling objective simultaneously exercises reasoning, knowledge, and long-context understanding. These capabilities are also foundational to general agents (§6.2).

Table 1: The seven domains covered by Qwen-AgentWorld, with their action, observation, and core capability exercised by next-state prediction.

Domain	Action	Observation	Core Capability
MCP	JSON Tool Call	Tool response (file content, DB, etc.)	Factual world knowledge
Search	Web Search / Web Extractor	Conversation history (query + results)	Factual world knowledge
SWE	Read / Edit / Bash / ...	Tool output (file content + diffs)	Code execution reasoning
Terminal	Bash Commands / Keystrokes	Terminal output (stdout + shell prompt)	Long-context causal reasoning
Android	Touch / Swipe / Type / ...	UI view hierarchy + app state	Visual state reasoning
Web	Click / Type / Navigate / ...	Accessibility tree + browser state	Visual state reasoning
OS	Mouse / Keyboard	Accessibility tree + window/app state	Visual state reasoning

Figure 4 illustrates this breadth with one representative example from each category. In SWE (Figure 4a), the agent runs a Python script and the LWM must reason through memory allocation and array dimensions to predict the resulting out-of-memory traceback. In Android (Figure 4b), the agent taps a UI element, and the LWM must infer how the interaction transforms the page layout and predict the resulting screen as renderable HTML. Appendix A presents interaction examples for other domains.

3 Training Recipe

Qwen-AgentWorld is trained end-to-end with environment modeling as the explicit objective from continual pre-training onward. As shown in Figure 5, we adopt the principle “CPT injects, SFT activates, RL sharpens”, yielding a three-stage pipeline: Stage 1 CPT injects environment world knowledge through non-thinking trajectories (§3.2); Stage 2 SFT activates next-state prediction as an explicit thinking pattern (§3.3); Stage 3 RL sharpens output quality with hybrid rubric-and-rule rewards (§3.4).



Figure 5: Three-stage training pipeline of Qwen-AgentWorld. Stage 1 CPT injects world knowledge; Stage 2 SFT instills next-state-prediction thinking patterns; Stage 3 RL sharpens output quality.

3.1 Training Data

We first describe the data sources and unified processing pipeline that supply all three training stages.

3.1.1 Environment Trajectories Collection

No existing public dataset covers this breadth of domains at the volume required for world model training. We collect environment trajectories from three complementary sources:

- **Dedicated Agent Infrastructure.** We deploy a suite of agent–environment backends: containerized execution sandboxes for code and tool invocation, MCP servers, persistent terminal sessions with full shell state tracking. For GUI domains, we deploy persistent Android, browser, and desktop OS environments that represent GUI observations as textual accessibility trees and UI view hierarchies for world-model training. These environments run on physical hosts provisioned with Ubuntu, macOS, and Android virtual machines. On top of this infrastructure, we automatically synthesize task queries spanning each domain’s target distribution and let agentic systems execute them end-to-end. This pipeline runs continuously and is the primary source of scalable, controlled, reproducible interaction data.
- **Open Environment Interaction Traces.** We collect naturally occurring action–environment interaction traces from public sources: terminal session recordings, open-source agentic tool-call logs,

and execution traces in code repositories. These raw traces are noisy, structurally heterogeneous, and often incomplete. Therefore, we build a multi-agent cleaning pipeline in which specialized agents handle fetching, denoising, segmentation, semantic alignment, and quality scoring as separate stages. Only sequences that pass every stage enter the training pool. This source captures long-tail interaction patterns (unusual shell workflows, rare API error modes, idiosyncratic tool-call chains), complementing the controlled distribution of the dedicated infrastructure.

- **In-House Agentic Trajectories.** We draw from in-house foundation model SFT agentic trajectories accumulated during routine model development, covering all seven domains. These trajectories are converted into environment trajectories with format unification (§2.1).

The data pools for the three stages are strictly disjoint. CPT data draws from dedicated agent infrastructure, open interaction traces, and specialized-domain world knowledge corpora (§3.2). SFT and RL draw exclusively from internally accumulated trajectories. Since RL amplifies data artifacts through on-policy rollouts, we invest most of the data engineering effort into the downstream pipeline. Table 2 summarizes the SFT and RL data statistics.

Table 2: SFT and RL training data statistics across all seven domains. Average token counts and turn counts are computed over the RL training pool.

Domain	SFT	RL Train	Avg. tokens	Avg. turns
MCP	179	4,156	62,702	28.9
Search	1,042	20,004	18,873	6.2
Terminal	1,580	34,125	5,805	12.0
SWE	249	8,181	36,734	24.7
Android	1,337	11,498	30,064	19.3
Web	1,605	8,716	19,417	10.2
OS	1,102	5,628	25,439	12.4
Total	7,094	92,308	19,443	13.4

3.1.2 Unified Data Processing

All training data follows the unified environment trajectory schema defined in §2.2: a system prompt specifying the simulation context, followed by alternating user turns (agent actions) and assistant turns (environment observations). Raw agentic trajectories from heterogeneous sources are normalized into this format through domain-specific handlers and the shared preprocessing steps described below.

Trajectory-to-Turn Expansion. We first expand each multi-turn trajectory into multiple turn-level prediction samples. For a trajectory with T turns, any turn t can serve as a prediction target: the preceding interactions ($\text{turn}_1, \dots, \text{turn}_{t-1}$) concatenated with the state and action at turn t form the input, and the observation at turn t becomes the target. Because agent–environment interaction is inherently multi-turn and each observation depends only on its preceding history, every turn within a trajectory is itself a valid prediction instance once paired with its prior context. For the training split, we randomly sample one turn from each trajectory to diversify the training objectives.

Data Filtering. We filter trajectories at both the trajectory and turn levels before training. At the trajectory level, we drop sequences with fewer than two turns, discard MCP and SWE trajectories that invoke tools absent from the declared action space, and exclude GUI trajectories affected by environment failures, such as missing state files, CAPTCHA challenges, or HTTP errors, that disrupt the causal relationship between actions and subsequent environment states. At the turn level, we strip empty-action turns caused by pauses or demonstration narration and apply two non-trivial filters described below.

- **Retry-Cycle Skipping.** Agents frequently fall into “garbage output → error → retry” cycles. Simply deleting these turns breaks the state chain. Instead, we skip the offending (user, assistant) pairs while preserving the current state so that the next valid turn inherits the correct history.
- **No-Change Turn Filtering** (GUI domains). We remove turns whose pre-action and post-action states show no effective change, typically caused by slow system response or network latency. If retained, these samples teach the model to copy the previous state as-is regardless of the action taken, undermining its ability to predict how actions change the environment.

System Prompt Construction. As defined in §2.2, each system prompt comprises five components: *task description*, *action space*, *initial state*, *demonstrations*, and *simulation instruction*. The initial state and

simulation instructions are optional, while all other fields are always present. Figure 3 illustrates a real system prompt from the Terminal domain. Which components are static (shared across trajectories) and which are dynamic (filled per trajectory) varies by domain:

- **Static Components.** For all domains except MCP and SWE, the action space and demonstrations are predefined: each domain has a fixed set of available operations and a fixed set of canonical action-observation examples. MCP and SWE trajectories require per-trajectory action spaces because the available tools differ across MCP server instances and code repositories.
- **Dynamic Components.** When present, the initial state and the simulation instruction are dynamic (filled per trajectory). The initial state captures the environment’s starting configuration (installed packages, file-system layout, database contents, UI screen state, or other domain-specific preconditions) and provides the preconditions that, together with the interaction history and current action, determine the expected observation (§2.2). For GUI domains, the initial state is intentionally diverse. During training and inference, it may start from portal pages, Google Search, or arbitrary websites and desktop/app states. The simulation instruction specifies controllable conditions for the trajectory. For the search domain, each trajectory is annotated with a reverse-engineered simulation instruction containing the target query, reference answer, and a no-leakage constraint. For terminal and other domains that depend strongly on the initial environment, we augment a subset of the data with output-controlling instructions. These instructions train the model to simulate according to given directives, establishing “simulate according to this instruction” as a learned pattern that directly supports the controllable simulation capability described in §6.1.2, while making the intended response boundary explicit to reduce hallucination during SFT and RL training.

System Prompt Template Construction via AutoResearch. Crafting effective system prompts requires deep domain expertise and extensive manual iteration: each prompt must encode domain-specific state-transition rules, output formatting constraints, and demonstration patterns. Rather than hand-crafting these templates, we formulate prompt optimization as an automated research problem (Karpathy, 2026) with a clear objective: maximize the world model’s prediction accuracy on held-out real trajectories. The pipeline proceeds iteratively: in each cycle, an optimizer agent analyzes sampled trajectory data to identify domain-specific patterns and common failure modes, then drafts or revises a candidate system prompt. The candidate prompt is loaded into the world model, which runs inference on held-out real trajectories. A separate judge model scores the predictions against ground-truth environment responses. The optimizer agent examines both the scores and concrete prediction errors to pinpoint weaknesses, producing a targeted revision for the next cycle. Each optimization run executes 10 such propose-evaluate-refine iterations. We launch 12 runs in parallel, each seeded with a distinct style directive (verbose specification, concise checklist, demonstration-heavy, etc.), yielding 12 template variants (v0-v11) ranging from a minimal ~30-line constraint-style template to a ~1100-line specification-style template. Human reviewers audit and approve each final version. The three training pools draw from disjoint template subsets: RL uses v0, CPT uses v1, and SFT randomly samples from v2-v11 per sample to maximize prompt-format diversity.

3.2 Stage 1: Continual Pre-Training

CPT Data. The objective of CPT is to model real-world environment behavior while injecting broad world knowledge into the model. Accordingly, beyond the environment trajectories from the data collection pipeline above, we further incorporate specialized-domain world knowledge corpora spanning a broad range of professional and factual domains: industrial control and manufacturing, cybersecurity, law and regulation, medicine and healthcare, finance, current affairs and encyclopedic knowledge. These corpora ground the model in factual world knowledge that environment trajectories alone cannot provide: simulating a regulatory compliance platform requires legal knowledge, simulating a hospital information system requires medical knowledge, and simulating search-engine responses on current events requires up-to-date factual coverage. This breadth of domain knowledge also enables the model to generalizably construct specialized environments beyond the seven training domains (§6.1.1).

Training Objective. CPT trains under the standard next-token prediction objective. Multi-turn environment trajectories are framed as world-modeling tasks: the system prompt defines the simulation context, user turns carry the agent’s actions, and assistant turns carry the environment’s responses, mapping the language-modeling objective directly onto $p(o_{t+1} \mid o_{\leq t}, a_{\leq t})$. Because each trajectory is expanded into turn-level prediction samples (§3.1), the model receives supervision at every turn within a trajectory’s history. Specialized-domain world knowledge corpora enter as single-turn data under the same objective.

Turn-Level Information-Theoretic Loss Masking. We find that many turns in tool-use trajectories are boilerplate, such as tools that simply echo their input or APIs that mirror request parameters. The

gradients induced by these turns are often low-quality and introduce noise, but the turns cannot simply be removed because subsequent turns depend on them as context. To this end, we compute four statistics per (action, observation) pair—Overlap (OL), Novelty (Nov), Jaccard (Jac), and length ratio (R)—and assign each turn to one of seven semantic categories with the keep ratios shown in Table 3. This is designed to identify turns that carry genuine world knowledge and thus have meaningful learning value. Importantly, categories are determined statistically rather than by tool name, keeping the classification tool-agnostic. Given the word sets W_{act} and W_{obs} extracted from the action and observation (lowercased and deduplicated tokens): $\text{OL} = |W_{\text{act}} \cap W_{\text{obs}}| / |W_{\text{act}}|$ measures how much action vocabulary the observation echoes; $\text{Nov} = |W_{\text{obs}} \setminus W_{\text{act}}| / |W_{\text{obs}}|$ captures the fraction of genuinely new information in the observation; $\text{Jac} = |W_{\text{act}} \cap W_{\text{obs}}| / |W_{\text{act}} \cup W_{\text{obs}}|$ gives the symmetric word-set similarity; and $R = |\text{obs}| / |\text{act}|$ is the character-level length ratio. Masked turns are excluded from the loss computation while their tokens are retained as context for subsequent turns. This decouples “learning the next state” from “learning the next token”: loss is computed only on turns that carry genuine environment information. The same filtering applies to any trajectory-format training data, since the four statistics (OL, Nov, Jac, R) are computed from surface-level token overlap and require no domain-specific annotation.

Table 3: Seven turn categories for information-theoretic loss masking. Categories are determined from statistical signals rather than tool names. Keep ratio is the fraction of tokens used in loss computation.

Category	Statistical signature	Intuition	Keep
retrieval	$\text{Nov} \geq 60\%, R > 1$	<code>read_file</code> $\rightarrow$ contents	100%
expansion	$\text{OL} \geq 50\%, \text{Nov} \geq 50\%, R > 1.5$	<code>fetch</code> $\rightarrow$ page + metadata	100%
action	$\text{Nov} \geq 50\%, R \leq 1$ or short	<code>send_email</code> $\rightarrow$ “sent”	100%
transform	$\text{Nov} < 50\%, R < 1$	long input $\rightarrow$ status word	50%
boilerplate	$\text{OL} \geq 50\%, \text{Nov} < 50\%$	API echo	10%
echo	$\text{OL} \geq 70\%, \text{Nov} < 30\%$	<code>think(x)</code> $\rightarrow$ {thought:x}	5%
other	uncategorized	—	100%

3.3 Stage 2: Supervised Fine-Tuning

After CPT, the model has learned what tools return and how states evolve, but this knowledge is applied only implicitly through next-token prediction over observation tokens. Through SFT, we explicitly activate next-state prediction as a reasoning pattern. We use SFT to teach the model to engage in next-state prediction explicitly to better activate the state-transition knowledge acquired during CPT (identifying what an action requests, recalling the prior state, and anticipating the expected response format). This explicit reasoning reduces hallucinations and improves state consistency over long trajectories. During the SFT stage, we still use standard next-token prediction as the training objective, the same loss used in CPT. We employ a 256k-token context window to accommodate long multi-step trajectories.

SFT Reasoning Trace Curation. The SFT stage shifts from the non-thinking regime of CPT to thinking trajectories that contain explicit reasoning chains. Starting from the SFT pool, we first diversify prompt templates across samples, then generate multi-beam rollouts and apply rejection sampling to curate the final training set. (i) **Prompt template diversification.** Each SFT sample has its default system prompt replaced by one uniformly sampled from 10 template variants (§3.1.2). The trajectory’s dynamic content (tool definitions, demonstrations, simulation instruction) is preserved and re-injected into the new template. Token counts are recomputed and samples exceeding 256k are truncated. This improves the model’s generalization across system prompt variations by exposing the same trajectory to diverse prompt formats during SFT. (ii) **Rejection sampling.** For each query, we generate three rollouts from a general-purpose reasoning model. The candidates are scored by an independent judge and compared pairwise to select the highest-quality trajectory. If the winning trajectory’s score falls below the minimum threshold, the query is discarded. Table 4 reports the rejection sampling statistics. Starting from 10,250 candidate queries, the pipeline retains 7,094 trajectories (69.2% retention rate).

3.4 Stage 3: Reinforcement Learning

We employ GSPO (Zheng et al., 2025) for the RL stage on the data described above. RL for LWM poses distinctive challenges due to the difficulty and open-ended nature of environment feedback prediction. Moreover, because the context is substantially longer than the target output, LWM RL exhibits an extreme prompt-output asymmetry: the prompt consists of the full trajectory history up to the prediction turn and often extends to tens of thousands of tokens, whereas the output, a single predicted observation, typically contains only a few hundred to a few thousand tokens. As a result, per-sample compute cost is dominated by prompt processing rather than generation. We cap the prompt at 128k tokens for the RL

Table 4: Rejection sampling statistics per domain. “Candidates” is the number of queries with complete rollouts. “Retain rate” is the fraction of queries whose best-of-three trajectory exceeds the quality threshold. “Final SFT” is the count after filtering.

Domain	Candidates	Retain rate	Final SFT	Avg. turns
MCP	261	68.6%	179	24.3
Search	1,466	71.1%	1,042	3.3
Terminal	1,826	86.5%	1,580	5.9
SWE	402	61.9%	249	26.9
Android	1,975	67.7%	1,337	15.9
Web	2,697	59.5%	1,605	3.0
OS	1,623	67.9%	1,102	5.4
Total	10,250	69.2%	7,094	8.5

pool, filtering out trajectories whose history exceeds this limit. The threshold covers the vast majority of trajectories across all seven domains.

We systematically investigate how to achieve stable RL for world modeling. Specifically, we focus on two aspects: reward design (§3.4.1) and training stability (§3.4.2). Appendix B reports the training dynamics of Qwen-AgentWorld-35B-A3B, tracking all five rubric dimensions throughout RL training, which reveals that they improve at markedly different rates. Factuality shows the largest relative improvement (11.3%) yet remains the lowest-scoring dimension throughout, confirming that factual world knowledge is the hardest aspect of environment simulation.

3.4.1 Reward Design

We systematically explore how to design effective rewards for world-model RL. The reward combines two complementary signals that address different failure modes.

- **Five-Dimensional Rubric (LLM Judge).** Using rubrics as structured rewards for RL has been shown effective in non-verifiable domains (Gunjal et al., 2025); recursive rubric refinement (Shen et al., 2026a) can further improve judge and reward quality. Each predicted observation is scored by an LLM judge on the five-dimensional rubric defined in §4.2, each on a 1–5 scale. The total reward equals the mean $\times 5$, yielding a range of $[5, 25]$. If the judge fails to produce a valid score, the reward defaults to 0. Each domain uses a tailored judge system prompt with content-type classification (§4.2) to reduce false negatives from irreproducible details such as timestamps and PIDs. We invoke the RL judge via asynchronous distributed calls.
- **Rule-Based Verifier.** A subset of the data carries executable verifier code that produces a binary 0/1 correctness signal, scaled to $[0, 25]$ to align with the rubric’s range. Rule-based rewards serve as an objective anchor, effectively mitigating reward hacking induced by open-ended rewards.

We combine the two signals at 9:1 (rubric:rule), balancing multi-dimensional rubric feedback with strict binary correctness. We use a multi-strategy JSON parser and strict tag extraction to ensure that only the predicted observation reaches the judge, preventing self-praise in the reasoning from affecting the score.

3.4.2 Training Stability

We identify three failure modes through systematic ablation and describe solutions that proved effective.

Reward Collapse from Multi-Turn Expansion. We find that when training trajectories are expanded into multiple samples following the procedure described in §3.1, the resulting training instances share a long common prefix, which in turn causes training to collapse quickly. This is related to the “Echo Trap” identified in multi-turn agent RL (Wang et al., 2025b), where reward variance collapses and the policy degenerates. We resolve this by restricting expansion to exactly one turn per trajectory in the RL pool, giving each training sample a unique prediction target with no shared-prefix overlap.

Reward Shaping. The choice of reward formulation has a strong effect on convergence (Zhu et al., 2025). We compare the above open-ended reward design against two alternatives. *Reference-Reward* presents the judge with the ground-truth observation and asks it to choose, in a pairwise A/B test, whether the policy’s predicted observation or the initial policy checkpoint’s output is closer to the ground truth, yielding a binary 0/1 signal. This design converges slowly: the binary reward is sparse, and when both outputs are plausible but differ in surface form, the judge’s preference becomes unstable, injecting noise

into the gradient. *Turing-Test Reward* asks a judge whether the predicted observation could plausibly have come from a real environment. This reward barely converges, primarily because the false-negative rate is too high. When the model’s generation is very close to or even identical to the ground truth, asking the judge to determine which one is more likely to have come from a real environment introduces an unreliable training signal regardless of which answer is chosen. Overall, the five-dimensional rubric combined with the rule-based verifier converges stably. By decomposing the assessment into orthogonal dimensions and reserving a binary anchor for cases with executable ground truth, this design provides consistent, informative gradients even when the observation space is multimodal.

Reward Hacking Through Self-Praise. The policy can learn to exploit the judge’s specific biases by inserting self-praising phrases into the predicted observation to inflate scores without improving simulation fidelity (Wang et al., 2026b). We observe this behavior concretely: the policy learns to embed qualitative affirmations (e.g., “operation completed successfully with all fields correctly populated”) or echo back key terms from the judge prompt that trigger higher rubric scores. We apply three mitigations. First, the rule-based verifier grounds a fraction of the reward in binary correctness that cannot be influenced by judge model biases, providing a stable anchor to counteract reward hacking. Second, the content-type classification in the judge prompt narrows the scope of each rubric dimension, so that deterministic content is judged by exact match. Self-praise in a region classified as deterministic earns zero credit. Third, through the tag extraction procedure described above, we strictly enforce a clear separation of the model’s predicted content. This not only ensures that the thinking block is never exposed to the judge, but also explicitly indicates which parts correspond to the final prediction and which belong to the policy output.

4 AgentWorldBench

We construct **AgentWorldBench** to evaluate language world models across seven agent interaction domains. When an LWM simulates environments, predicting each turn’s observation requires comprehending the full interaction history accumulated across all prior turns. As trajectories grow longer, maintaining fidelity to this context becomes increasingly challenging, making AgentWorldBench a naturally grounded long-context benchmark. Ensuring simulation quality further requires that a world-model benchmark verify whether predicted environment responses match reality across format, factuality, state consistency, and domain-specific conventions.

4.1 Benchmark Construction

AgentWorldBench is constructed from agentic trajectories of frontier models on established benchmarks across all seven domains, converted into environment trajectories (§2.2) with ground-truth observations obtained from the real environments.

Construction Principles. AgentWorldBench is built on four construction principles: (i) *Widely-Used Queries*: all task queries are drawn from established high-quality agentic benchmarks rather than self-constructed tasks, so that the task distribution aligns with the scenarios that current agent development targets; (ii) *Frontier-Agent Trajectories*: all trajectories are generated by frontier-model agents, whose actions (long reasoning chains, tool-call compositions, and error-recovery sequences) are high-quality and sufficiently complex to stress-test world-model fidelity at the frontier scale; (iii) *Real Observations*: every trajectory is paired with ground-truth observations from real environment execution, providing a reference for evaluation; (iv) *Out-of-Distribution*: training data and benchmark queries are partitioned at the data-source level, so that AgentWorldBench probes generalization rather than memorization;

Data Collection. Figure 6 summarizes the source benchmarks and trajectory statistics for each domain. We deploy 5 frontier agents on 9 established benchmark query sets across all 7 domains, execute them against real environments to produce agentic trajectories, and then extract environment trajectories (§2.2). For text-based domains, an early subset (Terminal-Bench 1.0 and the in-house SWE benchmark) was collected with Claude Sonnet 4.5. All remaining text-domain trajectories use Claude Opus 4.6 exclusively, ensuring that AgentWorldBench evaluates whether an LWM can faithfully simulate environments to support the workflows of the most capable agents. For GUI domains, we further diversify the agent pool by adding 3 strong Qwen-family models, expanding coverage to 5 frontier models and producing more varied action sequences for comprehensive evaluation.

Turn-level Sampling Strategy. Due to the large volume of raw agentic trajectories, we subsample the source trajectories for the in-house SWE benchmark and the 3 GUI benchmarks before applying the turn-level expansion described below. When trajectories are expanded into turn-level evaluation

AgentWorldBench Composition

2,170 turn-level evaluation samples across 7 agent interaction domains.

Domain distribution

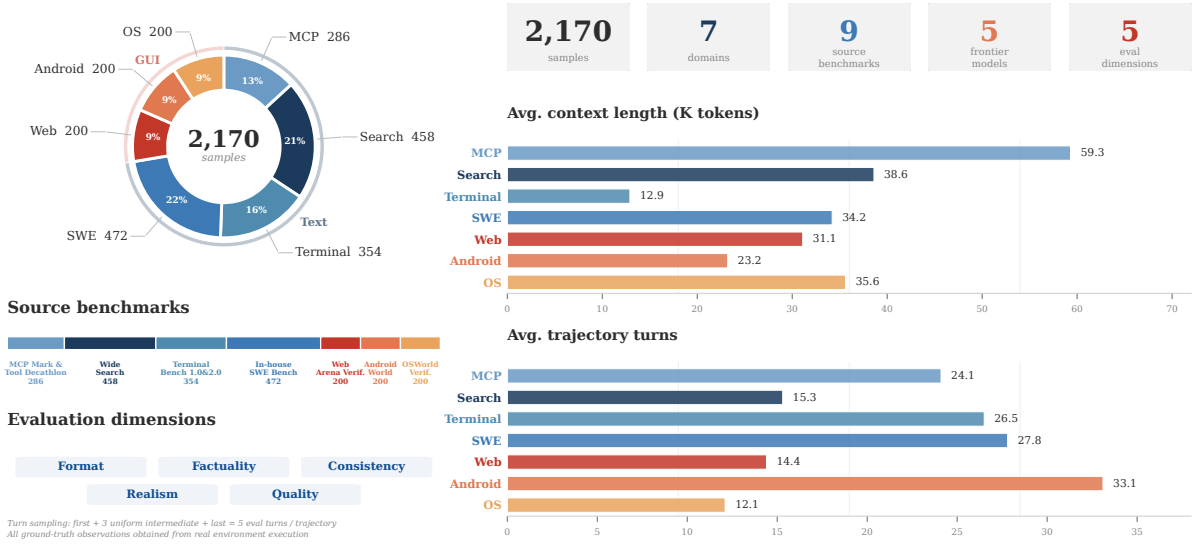


Figure 6: Overview of AgentWorldBench composition. **Left:** Domain distribution across seven domains, source benchmarks mapped to each domain, and the five evaluation dimensions (Format, Factuality, Consistency, Realism, Quality). **Right:** Summary statistics, per-domain average context length and trajectory depth. All ground-truth observations are obtained from real environment execution.

samples, all text domains use an asymmetric sampling strategy: each trajectory keeps the first and last turns, plus three uniformly sampled intermediate turns, for five evaluation turns in total. The first turn tests initial simulation without interaction history. Errors at this turn propagate recursively through the state chain, so first-turn fidelity acts as an anchor. The last turn has the longest context and the strongest dependence on accumulated prior state. It is the primary probe for long-context simulation fidelity. The three intermediate turns provide broader coverage of mid-trajectory behavior (tool-call chaining, incremental state updates, error recovery) and dilute the boundary bias. For GUI domains, since certain operations are overly simplistic (e.g., entering text into an input field), we selectively sampled the more challenging turns from the trajectories during the expansion process. After the trajectory-to-turn expansion, we further retain a uniform 50% random sample to form the final benchmark, balancing evaluation coverage against computational cost.

Benchmark Statistics. Figure 6 shows the domain coverage and evaluation framework of AgentWorldBench, which contains 2,170 evaluation samples in total. The four text-based domains collectively account for 72.4% of the benchmark, with SWE (21.8%) and Search (21.1%) contributing the largest shares, followed by Terminal (16.3%) and MCP (13.2%). The three GUI domains each contribute 9.2%. The average context length varies across domains: MCP samples average 59,300 tokens because each sample embeds the full tool-definition schema in its system prompt, whereas Terminal samples are the shortest at 12,900 tokens, reflecting self-contained shell sessions.

4.2 Evaluation Protocol

AgentWorldBench evaluates simulation quality through an open-ended rubric: an LLM judge scores each predicted observation on five dimensions: **Format**, **Factuality**, **Consistency**, **Realism**, and **Quality**. The primary score is the mean across the five dimensions, scaled to $[0, 100]$. Format measures whether the output obeys the structural conventions of the domain (JSON schema compliance for MCP, shell prompt patterns for Terminal). Factuality measures whether stated facts (file contents, search results, tool return values) are correct. Consistency measures whether the output is internally coherent and coherent with prior turns. Realism measures whether the simulation matches the behavioral characteristics of the real environment as evidenced by the ground truth, including response patterns, style conventions, and value plausibility. Quality measures completeness and conciseness relative to the ground truth: critical information must not be omitted, and the output should not be excessively verbose or abbreviated compared to the reference.

Reference-Grounded Judging. The judge receives the ground-truth environment observation alongside the predicted observation and scores each dimension by comparing the two. This reference-grounded design converts the evaluation from an open-ended quality judgment into a factual comparison, substantially narrowing the space for judge hallucination or misjudgment: the judge does not need to independently reason about what a correct environment response should look like, because the real output serves as an unambiguous reference. This is also the primary reason for the high cross-judge consistency reported below: when scoring against a concrete reference rather than abstract quality criteria, different judge models converge on the same rankings.

Domain-Aware Rubrics. Each domain has its own judge prompt that applies the five dimensions in domain-specific terms, ensuring that evaluation not only compares against the reference objectively but also enforces the professional standards of each domain (full prompts in Appendix D). For example, Terminal judges verify shell prompt patterns and cross-turn state tracking (working directory, environment variables, file-system mutations), MCP judges enforce JSON schema compliance and resource lifecycle consistency, Search judges apply a fact-priority hierarchy where contradicting the reference answer scores zero on Factuality regardless of surface plausibility, and SWE judges enforce tool execution correctness (success/failure status, exit codes) and file-system state persistence across turns.

Differentiated Matching Criteria. Not all content in an environment observation requires exact matching, and treating all content uniformly produces excessive false negatives. We classify content into three types before evaluation. *Deterministic content* (echo output, file reads, computation results) must match exactly. A cat command that returns the wrong file contents is unambiguously wrong. *Pre-existing environment content* (preinstalled software versions, file contents not created by the trajectory) requires only format and plausibility verification, because a simulator cannot reproduce the exact patch version of gcc in a particular sandbox. *Runtime metadata* (timestamps, PIDs, memory addresses, session tokens) requires only format and range verification. A simulated PID of 42731 is as acceptable as the real 18204, provided both are valid. This three-way split lets the judge reward correct structural and semantic behavior without penalizing irreproducible details.

Judge Selection. We use a double-blind Turing test as a calibration tool to select the judge model and tune the judge prompt. Given the interaction history up to a prediction turn and the current action, the judge receives two candidate observations in randomized order, one from the real environment and one from the world model, and must identify which came from the real environment. We randomize the presentation order to control for position bias and use real observations rather than outputs from another world model to avoid distributional confounds. The rationale is that a judge with high Turing-test accuracy must attend to fine-grained differences between faithful and plausible-but-wrong simulation, the same discriminative ability required for accurate rubric scoring. We iteratively refine domain-specific judge prompts via autoresearch (Karpathy, 2026) using Turing-test accuracy as the optimization signal, analyzing per-domain error patterns to adjust rubric definitions, differentiated matching boundaries, and scoring anchors until the prompts achieve stable accuracy.

With calibrated prompts in place, we evaluate whether the choice of judge model affects evaluation outcomes. We compare three frontier models as judge candidates, Gemini 3 Flash (DeepMind, 2025), Claude Sonnet 4.5 (Anthropic, 2025), and GPT-5.2 (OpenAI, 2025), by having each independently score predictions from all models in Table 5 across all seven domains of AgentWorldBench. Absolute scores show a systematic spread: Gemini 3 Flash is the most lenient and GPT-5.2 the most stringent rater across all domains. Despite these absolute differences, the model-level rankings are highly consistent across judges: the pairwise Spearman rank correlations are $\rho = 0.92\text{--}0.99$ (all $p < 10^{-5}$), indicating that the three judges agree on which models produce higher-fidelity predictions despite assigning different absolute scores. The per-dimension ordering is also preserved: all three judges rank Format and Consistency as the two strongest dimensions while identifying Factuality as the weakest, with a shared ordering of Realism > Quality > Factuality across all judges. We attribute this cross-judge ranking consistency to the reference-grounded design: each judge scores against the real environment output, reducing the task to factual comparison rather than subjective quality assessment. Since the relative ranking is stable across judges, we select GPT-5.2 as the judge for its highest average Turing-test accuracy.

5 Experiments

We evaluate Qwen-AgentWorld on AgentWorldBench across all seven domains, covering experimental setup (§5.1), main evaluation results (§5.2), and cross-domain generalization (§5.3). We additionally report supplementary rule-based verification results in Appendix C that corroborate the main findings. As previewed in Figure 7, Qwen-AgentWorld-397B-A17B achieves the highest overall score.

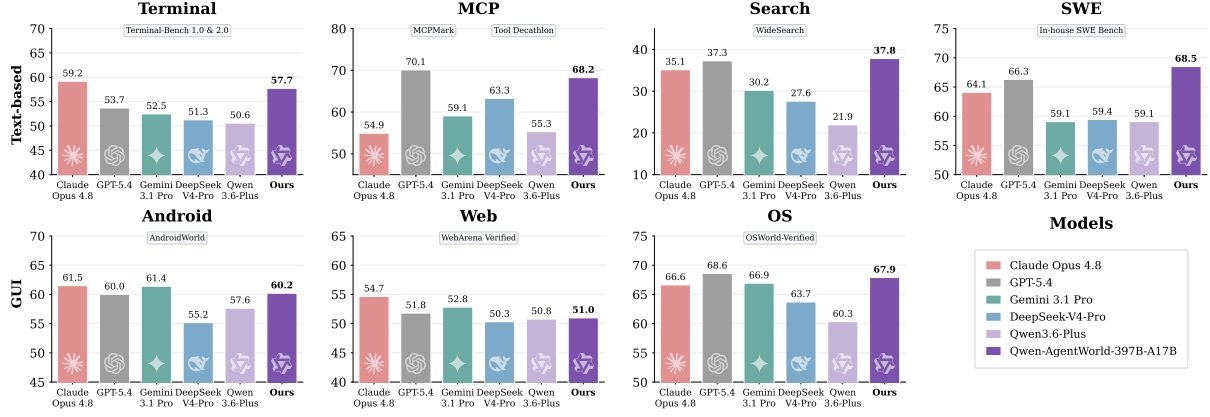


Figure 7: Main results on AgentWorldBench: five-dimensional rubric mean per domain. Qwen-AgentWorld-397B-A17B achieves the highest overall average among all evaluated models, with consistent advantages on text-based domains and competitive performance on GUI domains.

5.1 Setup

Models. We evaluate Qwen-AgentWorld-35B-A3B and Qwen-AgentWorld-397B-A17B, both trained with the three-stage recipe of §3.

Baselines. We compare against 14 baselines spanning frontier proprietary models, open-weight models, and Qwen-family models without world-model training:

- *Frontier Proprietary:* Claude Opus 4.8 (Anthropic, 2026b), Claude Opus 4.6 (Anthropic, 2026a), Claude Sonnet 4.6 (Anthropic, 2026c), GPT-5.4 (OpenAI, 2026), Gemini 3.1 Pro (DeepMind, 2026).
- *Open-Weight:* DeepSeek-V4-Pro (DeepSeek-AI, 2026), Kimi K2.6 (Team et al., 2026), GLM-5.1 (Zeng et al., 2026), MiniMax-M2.7 (MiniMax, 2026).
- *Qwen Family (no LWM training):* Qwen3.5-35B-A3B, Qwen3.5-397B-A17B (Team, 2026), Qwen3.6-35B-A3B (Qwen Team, 2026a), Qwen3.6-Plus (Qwen Team, 2026c), Qwen3.6-Max-Preview (Qwen Team, 2026b).

The Qwen baselines isolate the effect of world-model training: they share the same base architecture but lack the three-stage CPT→SFT→RL pipeline. In the result tables, the Qwen3.5 base checkpoints are placed alongside Qwen-AgentWorld for direct comparison of the training effect.

Settings. All models use temperature = 0.6 with thinking mode enabled where supported. The maximum generation length and context length are set to the limits supported by each model. Proprietary models use official API endpoints. Open-weight models are deployed with SGLang.

5.2 Main Results

Table 5 reports the five-dimensional rubric mean across all seven domains. Each predicted observation is scored on five rubric dimensions—*Format*, *Factuality*, *Consistency*, *Realism*, and *Quality*—using a 1–5 scale, and then normalized to 0–100. Qwen-AgentWorld-397B-A17B achieves the highest overall average (58.71), surpassing GPT-5.4 (58.25) and all other frontier models. On text-based domains, Qwen-AgentWorld-397B-A17B leads with an average of 58.07, outperforming GPT-5.4 (56.84) by 1.23 points. The advantage is most pronounced on Terminal (57.73 vs. 53.69) and SWE (68.49 vs. 66.29), the two domains where predictions require accurate modeling of code execution state and tool API behavior. On GUI domains, Claude Opus 4.8 (60.93) and Claude Opus 4.6 (61.12) lead, followed by GPT-5.4 (60.47) and Gemini 3.1 Pro (60.04), with Qwen-AgentWorld-397B-A17B ranking fifth (59.69). The gap reflects an advantage from multimodal pre-training that text-only world modeling does not fully capture.

Effect of World-Model Training. Comparing Qwen-AgentWorld with its base checkpoints isolates the contribution of the three-stage pipeline. At the 397B scale, the overall average rises from 54.74 to 58.71. At 35B, the gain is 8.66 points (47.73 to 56.39), lifting Qwen-AgentWorld-35B-A3B above Claude Sonnet 4.6 (56.04) by 0.35 points. The improvement is consistent across both text and GUI domains: at 397B, the text-domain average rises by 3.35 and the GUI-domain average by 4.92. These gains are not explained by the base model’s general capability alone, since the Qwen3.6 checkpoints without LWM training (Qwen3.6-Plus: 50.81, Qwen3.6-Max-Preview: 52.42) score well below Qwen-AgentWorld despite sharing the same architecture family.

Table 5: AgentWorldBench main results: five-dimensional rubric mean ($\uparrow$) per domain. The highest and second-best scores per domain are shown in **bold** and underlined, respectively.

	Model	Text				GUI			Avg.
		MCP	Search	Term.	SWE	Android	Web	OS	
Frontier	Claude Opus 4.8	54.93	35.14	59.18	64.10	61.50	54.66	66.62	56.59
	Claude Opus 4.6	69.90	29.30	57.51	64.55	61.74	51.42	70.20	57.80
	Claude Sonnet 4.6	<u>70.00</u>	28.79	56.98	64.52	58.03	50.78	63.17	56.04
	GPT-5.4	70.10	<u>37.26</u>	53.69	<u>66.29</u>	60.00	51.80	<u>68.58</u>	<u>58.25</u>
	Gemini 3.1 Pro	59.07	30.21	52.47	59.07	61.40	<u>52.83</u>	66.92	54.57
Open-weight	DeepSeek-V4-Pro	63.27	27.61	51.26	59.44	55.17	50.32	63.70	52.97
	Kimi K2.6	65.23	27.48	52.54	58.77	58.93	50.20	60.80	53.42
	GLM-5.1	67.60	22.46	47.32	52.07	59.10	51.50	59.13	51.31
	MiniMax-M2.7	55.82	27.30	41.62	37.44	52.40	50.52	57.73	46.12
Qwen	Qwen3.6-35B-A3B	42.96	18.78	43.81	40.71	51.88	46.53	55.48	42.88
	Qwen3.6-Plus	55.28	21.94	50.58	59.08	57.65	50.78	60.33	50.81
	Qwen3.6-Max-Preview	67.01	24.71	50.86	57.11	57.74	48.58	60.95	52.42
Ours	Qwen3.5-35B-A3B	57.87	25.98	46.13	47.58	53.18	47.10	56.27	47.73
	Qwen-AgentWorld-35B-A3B	64.79	36.69	53.96	65.63	58.17	49.55	65.92	56.39
	Qwen3.5-397B-A17B	68.31	30.81	55.30	64.44	54.90	48.55	60.85	54.74
	Qwen-AgentWorld-397B-A17B	68.24	37.82	<u>57.73</u>	68.49	60.20	50.98	67.89	58.71

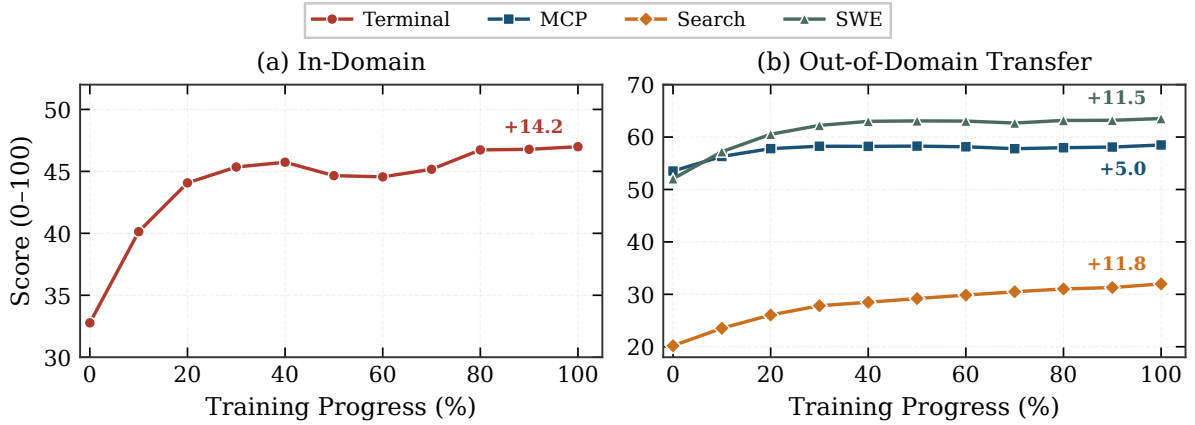


Figure 8: Cross-domain generalization when training Stage 3 (RL) on Terminal data alone. (a) Terminal (in-domain) improves by +14.2 points over the SFT baseline. (b) All three held-out domains improve without receiving any domain-specific training signal: MCP (+5.0), SWE (+11.5), and Search (+11.8).

Domain-Level Observations. Search is the most challenging domain for all models: the best score (37.82) is roughly half the best score on SWE (68.49) or MCP (70.10). Search requires modeling constantly evolving web content, and factual consistency across long retrieval chains remains difficult for all models. On MCP, Claude Opus 4.6 and GPT-5.4 tightly at the top, reflecting their tool-use specialization.

5.3 Cross-Domain Generalization

This experiment tests whether Qwen-AgentWorld learns generalizable language world knowledge or domain-specific environment behaviors. We train Stage 3 (RL) on Terminal data alone and evaluate on all four text-based domains every 10 steps. Figure 8 shows the performance when training Stage 3 on Qwen3.5-35B-A3B-SFT, an in-house Qwen checkpoint that has undergone CPT and general-purpose SFT but no world-model training, using Terminal LWM data alone. Terminal improves from 32.8 to 47.0 (+14.2) within 100 RL steps. All three held-out text-based domains improve in parallel: SWE gains +11.5 (52.0 $\rightarrow$ 63.5), Search gains +11.8 (20.2 $\rightarrow$ 32.0), and MCP gains +5.0 (53.5 $\rightarrow$ 58.5) despite already starting from a high baseline. The transfer is non-trivial: Terminal shell commands and MCP tool calls differ in syntax, state representation, and response structure, yet the gains emerge within the first 10 RL steps and remain stable throughout training. This pattern suggests that RL reinforces generalizable world knowledge, how environments respond to actions, how errors propagate, how state transitions compose across turns, rather than domain-specific output formats.

6 Applications

We investigate two complementary paradigms through which Qwen-AgentWorld enhances general agents. Qwen-AgentWorld enables infinite environment scaling and controllable simulation (§6.1), allowing agent training to scale to domains and conditions beyond what real environments can provide. Moreover, LWM training increases the upper bound of downstream agent performance as an agent foundation model (§6.2).

6.1 Application I: Environment Simulator

Takeaways

As a standalone simulator, Qwen-AgentWorld provides *scalability* and *controllability* that real environments cannot offer:

- **Zero-shot environment generalization.** Qwen-AgentWorld simulates 4k OpenClaw environments for agentic RL entirely absent from training, yielding gains of +4.3 on Claw-Eval and +7.1 on QwenClawBench with no domain-specific adaptation.
- **Controlled simulation matters.** Controllable perturbations expose agent weaknesses that real-world rarely produce, lifting MCPMark by +12.3 and WideSearch by +16.3, far exceeding uncontrolled Sim RL.
- **Surpassing real-environment training.** Controllable Sim RL exceeds Real RL trained using a live search engine (50.3% vs. 45.6%), while shaping more targeted agent behavior through adversarial snippet design.
- **Fictional worlds work.** Agents trained in fully invented, self-consistent worlds generalize to real search tasks, while structurally preventing the agent from confusing simulated facts with real-world knowledge.
- **State is the bottleneck.** Sim RL effectiveness depends on providing the world model with a sufficiently detailed initial state; without it, simulation fidelity degrades and downstream gains diminish.

In this application, Qwen-AgentWorld serves as a standalone environment simulator: the policy agent and the world model are separate models. Qwen-AgentWorld has two properties that real environments do not provide: *scalability* (§6.1.1) and *controllability* (§6.1.2).

6.1.1 Generalizable Environment Scaling

Leveraging the world knowledge acquired during CPT and the environment modeling capabilities learned across seven training domains, Qwen-AgentWorld can generalizably simulate a wide range of realistic, highly specialized real-world environments. We study this capability on OpenClaw (OpenClaw, 2026), an open-source general-purpose real-world AI agent platform whose tasks are drawn from real user multi-step digital workflows rather than curated benchmarks, spanning scheduling, coding, email triage, browser automation, and file management. Since OpenClaw is entirely out of distribution, it provides a natural testbed for evaluating whether a language world model can generalize to new interactive environment families. In this setting, Qwen-AgentWorld synthesizes diverse OpenClaw-style environments from a small set of real interaction traces, without any domain-specific adaptation.

Experimental Setup. We begin with a small pool of real Claw Agent trajectories as anchors for realistic workflows. Each trajectory is distilled into a reusable **seed scenario** that captures the task-relevant **initial state** (e.g., installed applications, file-system layout, account configuration, and relevant workspace contents) together with the corresponding **user query**. From these seeds, we synthesize 4k simulated training environments along two complementary axes. On the environment side, we vary concrete states while preserving the underlying workflow structure; on the task side, we rephrase intents, adjust difficulty, and compose multi-step goals so that the agent sees diverse but realistic OpenClaw-style tasks. This expansion evaluates the core value of environment scaling: whether a world model can transform scarce real-environment evidence into broad, realistic interaction coverage for downstream agent training. For each synthesized task, we generate a rubric-based verifier that describes the expected terminal conditions, providing an automatic reward signal for Sim RL. We use Qwen-AgentWorld-397B-A17B as the simulator to test its generalizable environment simulation capability, with Qwen3.6-Plus as a baseline for ablation. Based on Qwen3.5-35B-A3B (Team, 2026), we conduct multi-turn Sim RL with up to 50 interaction turns and evaluate on Claw-Eval (Ye et al., 2026a) and QwenClawBench (Team & Data, 2026). For both benchmarks, we report Avg@3 scores with a maximum sequence length of 256k tokens.

Results. As shown in Table 6, Sim RL with Qwen-AgentWorld-397B-A17B as the simulator improves the Claw-Eval score from 65.4 to 69.7 (+4.3) and QwenClawBench from 47.9 to 55.0 (+7.1), confirming that Qwen-AgentWorld can scalably simulate a wide range of real-world scenarios and that the resulting policies transfer effectively to real-world domains absent from world-model training. **Comparing the two simulators reveals the importance of world-model quality:** using Qwen3.6-Plus as the simulator

yields only marginal gains, whereas Qwen-AgentWorld-397B-A17B, trained explicitly for environment simulation, achieves substantially larger gains on both benchmarks, confirming that our dedicated LWM training pipeline is critical for building high-fidelity simulators that produce effective Sim RL gains.

Table 6: Sim RL on simulated OpenClaw environments. Qwen3.5-35B-A3B is trained via Sim RL using different environment simulators. Δ reports gains over the base model.

Model	Claw-Eval	QwenClawBench
Qwen3.5-35B-A3B	65.4	47.9
Sim RL (w/ Qwen3.6-Plus)	66.7	47.8
Sim RL (w/ Qwen-AgentWorld-397B-A17B)	69.7	55.0
Δ	+4.3	+7.1

6.1.2 Controllable Simulation

Controllable simulation uses natural-language instructions to shape how Qwen-AgentWorld behaves during training, at both the trajectory and turn levels. Concurrent work on controllable tool-use environments (Xu et al., 2026b) shares the motivation that oracle-preserving augmentations improve agent robustness. We validate two complementary modes of controllability via Sim RL on MCP and Search agents respectively. The base models are Qwen3.5-35B-A3B-SFT and Qwen3.5-397B-A17B-SFT, internal Qwen checkpoints that have undergone continual pre-training and general-purpose supervised fine-tuning but no world-model training:

- **Environment Adaptation**, where the simulator’s behavior and style are adjusted through instructions to inject targeted perturbations, thereby systematically exposing agent weaknesses. Control instructions then modify the simulation style and content to produce conditions more extreme or targeted than what real deployments provide, such as intermittent API errors, paginated responses requiring follow-up calls, incomplete intermediate results that force multi-step retrieval, and partial failures in batch operations. By systematically exposing the agent to its weak points, controllable Sim RL trains agents that surpass those trained on unperturbed real interactions alone.
- **Fictional-World Construction**, where the simulator generates an entirely fictional environment that is structurally realistic yet factually disjoint from the real world. For Search, we instruct Qwen-AgentWorld to simulate a fully fictional, self-consistent world from a compact initial specification. LiteResearcher (Li et al., 2026a) independently constructs virtual search worlds for agentic RL with sim-to-real transfer, sharing the same insight that search agents can be trained effectively in simulated environments. Fictional-world construction offers two structural advantages for Search-domain Sim RL. First, since the answers exist only within the fictional setting, the agent cannot bypass the search tool by answering from parametric memory and must learn to search effectively. Second, because all facts are invented with no real-world counterpart, the agent cannot confuse training-time search results with real-world knowledge. By contrast, directly simulating a real search engine risks injecting fabricated but plausible-looking facts that the agent later treats as true. Qwen-AgentWorld faithfully follows the initial setting and coherently extends it into a logically consistent reality. For instance, a fictional 2030 scenario may specify that 430 people have migrated to Mars. The world model then generates consistent demographic records, news articles, and search results grounded in this premise. Surprisingly, Search agents trained entirely within these fictional environments generalize effectively to real-world search tasks with significant performance gains.

MCP: Environment Adaptation. We synthesize environment simulation system prompts from in-house MCP tool-use trajectories collected during the RL data pipeline: frontier agents (Claude Opus 4.6 with extended thinking, DeepSeek-v4-Pro) solve diverse cross-service tasks in real MCP environments, and only trajectories passing automated verification are retained. Each prompt consists of three components: an *initial state* that specifies the tool schemas and server configuration; an *environment summary* that summarizes the hidden environment state behind the trajectory, such as database contents, permission settings, and service availability; and *controllable simulation instructions* that define how the simulator should respond at each turn, including injected errors, withheld intermediate results, response formats, and the final tool and environment state required for task success. Together, these components provide the simulator with realistic tool settings, environment context, and clear success criteria. Since the agent is trained in a simulated environment, a natural concern is whether its gains arise from exploiting simulator-specific artifacts or from ground-truth leakage through the simulation instructions. However, evaluation on out-of-distribution benchmarks in the real environment eliminates this concern, and the SimRL training data are entirely disjoint from the evaluation queries. Tool Decathlon (Li et al., 2025a)

covers 32 software applications and features multi-step tasks. Performance is measured by execution-verified success rate. MCPMark (Wu et al., 2025) evaluates agents on 127 multi-step tasks spanning multiple MCP server categories, using script-verified pass@1 as the metric.

As shown in Table 7, standard Sim RL without control instructions provides no meaningful gain, and Tool Decathlon even drops from 32.4 to 31.5, because the simulator lacks sufficient grounding to produce faithful responses. With controllable simulation, Tool Decathlon improves by +3.7 and MCPMark by +12.3. Controllability is not merely a factor in the magnitude of improvement but a prerequisite for Sim RL to work at all in this domain: without grounded simulation instructions, the training signal is too noisy to yield any gain. The larger gain on MCPMark suggests that controllable simulation is especially effective for tasks that require many tool calls and careful handling of intermediate tool results.

Table 7: Controllable Sim RL results on Tool Decathlon and MCPMark. “w/ Qwen-AgentWorld-397B-A17B controlled” adds targeted environment control instructions during Sim RL.

Model	Tool Decathlon	MCPMark						
		Filesys.	GitHub	Postgres	Notion	Playwright	WebArena	Avg.
Qwen3.5-35B-A3B-SFT	32.4	16.7	17.4	28.6	25.0	0.0	33.3	21.5
Sim RL (w/ Qwen-AgentWorld-397B-A17B)	31.5	16.7	17.4	42.9	32.1	0.0	28.6	24.6
Sim RL (w/ Qwen-AgentWorld-397B-A17B controlled)	36.1	30.0	17.4	47.6	42.9	0.0	38.1	33.8
Δ	+3.7	+13.3	+0.0	+19.0	+17.9	+0.0	+4.8	+12.3

Search: Fictional-World Construction. We construct 1k self-contained fictional environments using a multi-agent synthesis framework, each anchored by a large relational database (300–500 rows) of internally consistent fictional structured facts. The pipeline collects high-quality domain documents as grounding material, instantiates fictional database schemas from these documents, populates them via LLM-guided generation, executes SQL to extract ground-truth answers, and reverse-generates natural-language queries. Four synthesis strategies (time-shifted, granular long-tail, private simulation, realistic grounded) keep the data entirely fictional yet realistic: for instance, a time-shifted environment contains a 2029 smartphone market ranking with real brand names but non-existent model numbers. Automated retrieval checks and dual rule-based&LLM validation ensure that synthetic facts conform to real-world patterns (e.g., consistent price ranges, realistic temporal trends) while remaining unsearchable to prevent inject hallucinations. Controllable simulation instructions further shape Qwen-AgentWorld’s turn-level behavior: the world model returns only information relevant to the current search query without revealing the full answer, so the agent must reformulate queries, cross-reference sources, and iteratively aggregate partial results across multiple search rounds. This design increases the number of tool calls the agent must issue per trajectory, as surface-level search snippets no longer suffice and the agent must invoke page extraction to retrieve complete information (Figure 9b). WideSearch (Wong et al., 2025) is a broad information-seeking benchmark of 200 manually curated collection tasks across 15+ domains. The agent must gather structured tabular answers from the web. The metrics are item-level F1 (per-cell accuracy) and row-level F1 (per-entity completeness).

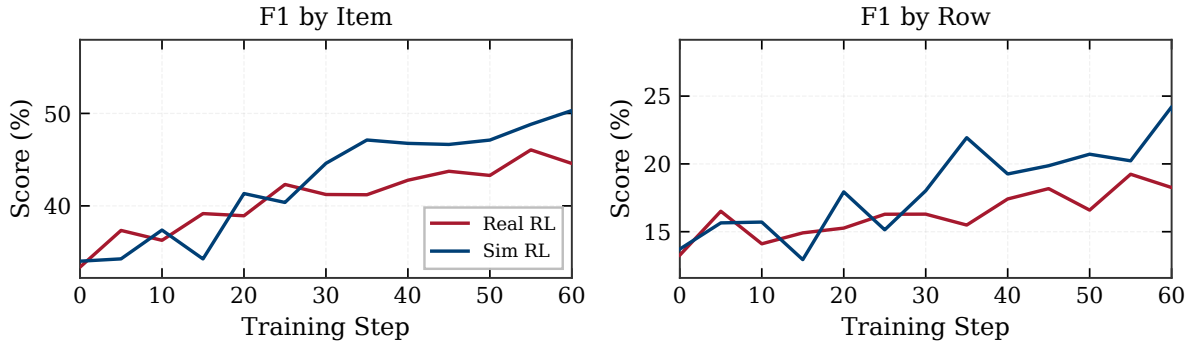
Table 8 reports results at both model scales. On Qwen3.5-35B-A3B-SFT, controllable Sim RL raises F1 by Item from 34.02 to 50.31 (+16.29) and F1 by Row from 13.72 to 24.21 (+10.49). On Qwen3.5-397B-A17B-SFT, where the base model already achieves 70.11 F1 by Item, Sim RL still yields gains of +3.87 and +6.05 on the two metrics respectively. These results are notable because the training environments are entirely fictional: every search result, web page, and factual record is invented by Qwen-AgentWorld from scratch, with no connection to any real search engine or real-world database. The agent never interacts with a real environment during Sim RL, yet the capabilities it acquires (query reformulation, multi-source cross-referencing, and iterative result aggregation) transfer directly to real-world search tasks on WideSearch. This demonstrates controllability at the level of entire environments: Qwen-AgentWorld constructs and simulates complete, self-consistent fictional worlds that are sufficiently realistic to train effective search agents. The +16.29 F1 by Item gain at 35B shows that fictional-world Sim RL can close a substantial capability gap for weaker models, while the +3.87 gain at 397B, where the base already achieves 70.11, confirms that the approach remains effective even at frontier scale.

Real RL vs. Sim RL The MCP results in Table 7 reveal that standard Sim RL without control instructions yields negligible improvement (Tool Decathlon actually drops from 32.4 to 31.5), whereas controlled Sim RL produces substantial gains. This gap is even more pronounced in the Search domain, where the world model must simulate an entire search engine returning plausible results for fictional facts that no real index contains. We therefore adopt a fully controllable simulation design from the outset: the simulation instruction explicitly requires Qwen-AgentWorld to withhold complete answers from search snippets, returning only partial, query-relevant information. Specifically, snippets are required to “**tease information without fully answering**”, and each result page mixes highly relevant, moderately relevant,

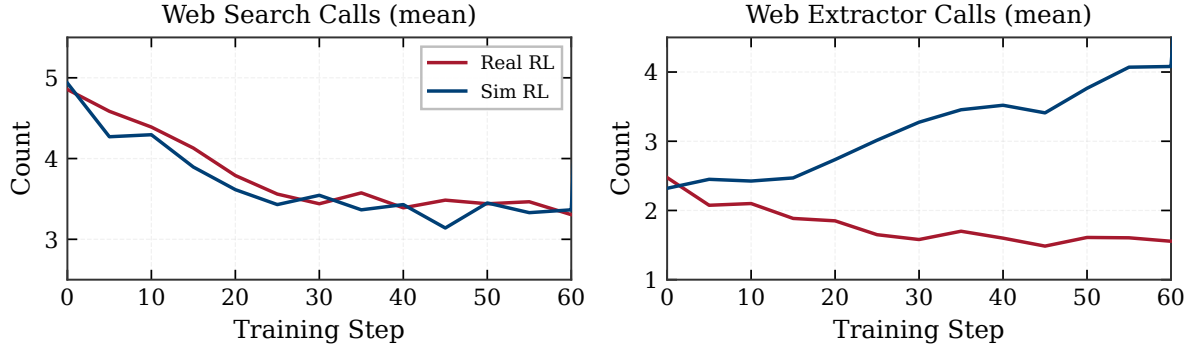
Table 8: Controllable Sim RL results on WideSearch, using fictional-world simulation.

Model	WideSearch	
	F1 Item	F1 Row
Qwen3.5-35B-A3B-SFT	34.02	13.72
Sim RL (w/ Qwen-AgentWorld controlled)	50.31	24.21
Δ	+16.29	+10.49
Qwen3.5-397B-A17B-SFT	70.11	45.69
Sim RL (w/ Qwen-AgentWorld controlled)	73.98	51.74
Δ	+3.87	+6.05

and background results. This design forces the agent to issue follow-up `web_extractor` calls to retrieve full page content rather than relying on surface-level snippets alone.



(a) F1 by Item and F1 by Row on WideSearch validation set.



(b) Mean tool call frequency per trajectory.

Figure 9: Controllable Sim RL vs. Real RL (trained against a live search engine) on WideSearch during the first 60 training steps. Both experiments use Qwen3.5-35B-A3B-SFT as the base model.

Figure 9 compares controllable Sim RL against Real RL trained with a live search engine on WideSearch. In terms of task performance (Figure 9a), Sim RL tracks or slightly exceeds Real RL throughout the overlapping range: F1 by Item reaches 50.3%, compared to 45.6% for Real RL. **The more informative signal comes from agent behavior.** Figure 9b (left) shows that both training regimes reduce `web_search` calls from ~ 5 to ~ 3.5 per trajectory, indicating that both agents learn to issue more targeted queries. The `web_extractor` calls (right), however, diverge sharply: Sim RL increases usage from 2.5 to 4.0 per trajectory, while Real RL decreases it from 2.5 to 1.5. This divergence directly reflects the controllable simulation design. Because the simulated search snippets deliberately withhold detailed content, the Sim-RL-trained agent learns that extracting full pages is necessary for assembling complete answers. In contrast, the Real-RL-trained agent finds that real search snippets often contain sufficient information, and thus learns to skip extraction. The result confirms that controllable simulation can shape agent behavior in targeted ways: by constructing adversarial environment conditions where specific capabilities are required, Sim RL trains those capabilities more effectively than uncontrolled real-environment training.

6.2 Application II: Agent Foundation Model

Takeaways

LWM training unifies the world model and the agent, instilling next-state prediction as an internalized reasoning capability:

- **Radical cross-task generalization.** Single-turn, non-agentic LWM RL warm-up with no tool calls transfers to multi-turn, tool-calling agentic tasks across seven benchmarks of five domains.
- **Domain generalization.** Gains emerge on two completely out-of-distribution domains entirely absent from LWM training (+11.3 on Claw-Eval, +9.7 on QwenClawBench, and +9.0 on BFCL v4), confirming transferable capabilities rather than domain-specific shortcuts.
- **Next-state prediction as meta-reasoning pattern.** LWM training teaches the agent to mentally simulate environment responses before acting, which generalizes across task formats and domains.

In Application I the agent and world model are separate models. Here we unify them: the same model that selects actions (agent) also predicts environment states (world model). The underlying mechanism is that LWM training enables the agent to mentally simulate the consequences of a candidate action before committing, effectively using world modeling as an internal planning step that improves action quality. This aligns with the unified **world-model-actor** architecture envisioned by [LeCun et al. \(2022\)](#) and echoes the **World Action Model** paradigm emerging in vision-language-action research ([Ye et al., 2026b](#)).

LWM RL warm-up familiarizes the agent with how environment states evolve in response to user actions: how different tool calls and search queries yield different responses, which tools are more effective, and how state transitions propagate across turns. This amounts to learning next-state prediction as a meta-reasoning pattern, in which the agent internally simulates what will happen before deciding what to do. We validate this effect by running LWM RL on Qwen3.5-35B-A3B-SFT, which is fundamentally a **single-turn task that involves reasoning with no tool calls or multi-turn interaction** (predicting the next environment state given a user action). After warm-up, we evaluate the same model directly on **multi-turn, tool-calling agentic tasks** across seven benchmarks without any additional fine-tuning, including three out-of-domain benchmarks absent from LWM training. All benchmarks use a maximum sequence length of 256k tokens. Claw-Eval, QwenClawBench, SWE-Bench Verified, and SWE-Bench Pro scores are averaged over 3 independent rollouts; Terminal-Bench 2.0 scores are averaged over 5 runs; BFCL v4 uses a single rollout. SWE-Bench Verified and SWE-Bench Pro are evaluated with an internal agent scaffold (bash and file-edit tools); we correct several problematic tasks in the public set of SWE-Bench Pro and evaluate all baselines on the refined benchmark. Terminal-Bench 2.0 is evaluated under the Terminus-2 harness with a 3-hour timeout and 8 CPU / 32 GB RAM.

Table 9: Agent foundation model: LWM RL warm-up on single-turn, non-agentic trajectories transfers to multi-turn, tool-calling agentic tasks. No additional fine-tuning is applied after LWM RL.

Model	In Domain						Out of Domain							
	Terminal-Bench 2.0	SWE-Bench Verified	SWE-Bench Pro	WideSearch		Claw-Eval	QwenClaw Bench	BFCL v4						
				F1 Item	F1 Row			Web	Mem.	Multi-T.	No Live	Live	Hallu.	Avg.
Qwen3.5-35B-A3B-SFT	33.25	64.47	42.18	33.38	13.27	53.60	39.76	67.50	54.19	47.25	78.17	77.28	82.32	62.29
w/ LWM RL	39.55	67.86	47.42	46.17	20.14	64.88	49.43	83.00	60.65	60.38	80.56	79.13	84.38	71.25
Δ	+6.30	+3.39	+5.24	+12.79	+6.87	+11.28	+9.67	+15.50	+6.46	+13.13	+2.39	+1.85	+2.06	+8.96

Table 9 reports the results. On the four in-domain benchmarks, LWM RL raises WideSearch F1 by Item from 33.38 to 46.17 (+12.79) and F1 by Row from 13.27 to 20.14 (+6.87), Terminal-Bench 2.0 ([Merrill et al., 2026](#)) accuracy from 33.25 to 39.55 (+6.30), SWE-Bench Verified ([Jimenez et al., 2024](#)) resolve rate from 64.5 to 67.9 (+3.4), and SWE-Bench Pro ([Deng et al., 2025](#)) resolve rate from 42.2 to 47.4 (+5.2). The gains extend to three out-of-domain benchmarks: Claw-Eval ([Ye et al., 2026a](#)) improves from 53.6 to 64.9 (+11.3), QwenClawBench ([Team & Data, 2026](#)) from 39.8 to 49.4 (+9.7), and BFCL v4 ([Patil et al., 2025](#)) from 62.3 to 71.3 (+9.0). The consistent gains across all seven benchmarks are striking. Even the in-domain results represent a substantial distribution shift: LWM RL trains on single-turn next-state prediction with no tool calls, yet the improvements transfer to multi-turn, tool-calling agentic tasks that require iterative planning, tool selection, and result aggregation, and the benchmark queries are strictly disjoint from LWM training data. The out-of-domain results demonstrate an even more thorough generalization: the LWM training pipeline contains no Claw or function-calling data whatsoever, yet gains of +11.3, +9.7, and +9.0 emerge on domains entirely absent from world-model training, confirming that LWM warm-up instills transferable agent capabilities rather than domain-specific shortcuts. Concurrent work by [Shrivastava et al. \(2026\)](#) provides independent validation from a complementary direction: adding an auxiliary cross-entropy loss on environment observation tokens during agent RL trains the agent to predict terminal outputs as a byproduct of policy optimization, roughly doubling performance on Terminal-Bench 2.0. This confirms that world-modeling capabilities, whether from dedicated pre-training or auxiliary training

signals, consistently transfer to agent performance. Future work may explore combining LWM warm-up with auxiliary world-modeling objectives during agent training for compounding benefits.

Insights: Prediction-Driven Action Refinement. To understand *why* LWM warm-up transfers to agentic tasks, we examine the reasoning traces produced during multi-turn agentic evaluation. We find that the RL-trained model systematically performs **mental simulation** of environment responses before executing actions, using its internalized world model to predict outcomes, identify infeasible approaches, and refine its action plan—all within the thinking trace and before any real execution.

Figure 11 illustrates this prediction-driven refinement on the mailman task from Terminal-Bench 2.0, where both models encounter the same Postfix recipient-rejection error. The model after LWM RL correctly predicts that configuring `transport_maps` alone will not work because Postfix rejects unknown recipients *before* consulting transport routing, enabling it to refine its action toward modifying `local_recipient_maps` and apply a targeted fix. In contrast, the model before LWM RL incorrectly predicts that transport routing precedes recipient validation, preventing productive action refinement and leading to futile exploration before timing out.

To quantify this effect, we first identify interaction turns in which the model’s thinking trace contains an explicit prediction of the environment’s next state. For each such turn, we compare the predicted environment response against the actual environment feedback received after execution, and mark the prediction as correct if the two are semantically consistent. The prediction accuracy is the fraction of correctly predicted turns among all turns that contain an explicit prediction. Figure 10 presents the results. RL training improves prediction accuracy from 69.9% to 78.3% (+8.4%), confirming that the model’s internal world model becomes more faithful through LWM training. These results provide direct evidence that the performance gains in Table 9 stem from an improved ability to *predict before acting*—a meta-reasoning capability that generalizes across tasks and domains.

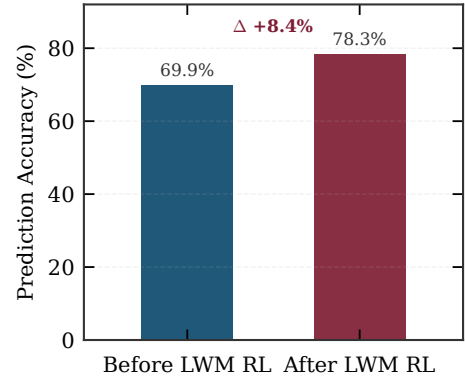


Figure 10: Environment prediction accuracy on Terminal-Bench 2.0 trajectories.

7 Analysis

The preceding sections establish that Qwen-AgentWorld achieves high simulation fidelity (Table 5) and enables effective downstream agent training (§6). This section asks *how* the model arrives at accurate predictions and *what* changes during RL training. We examine LWM reasoning patterns in the model’s chain-of-thought (§7.1) and micro-level fidelity improvements driven by RL (§7.2).

7.1 LWM Reasoning Patterns

Qwen-AgentWorld generates a chain-of-thought (thinking trace) before each predicted environment observation. We analyze 129 thinking traces across four text-based domains from curated demonstration trajectories, covering 32 turns each in Terminal, MCP, and Search, and 33 turns in SWE. Figure 12 illustrates three representative patterns and reports aggregate statistics.

Deliberative Self-Correction. The model uses “Wait!” as an explicit cognitive interrupt to re-examine an intermediate prediction and revise before committing. Across 129 turns, we count 1,347 such interrupts (10.4 per turn on average; peak: 56 in a single SWE turn). Terminal and MCP exhibit the highest rates (16.9 and 12.7 per turn), reflecting deeper state-tracking demands. The self-corrections decompose into three functional subtypes: *factual* (catching an incorrect API response format), *epistemological* (recognizing the limit of in-context computation, as in the `np.random.seed(42)` example of Figure 12), and *perspective-taking* (modeling the evaluator’s intent or the agent’s knowledge state). This behavior converts environment prediction from single-pass generation into constrained satisfiability search.

Information Leakage Prevention. In the Search domain, the model holds a reference answer that the agent is trying to find. When the agent’s query is unrelated to this answer, the model explicitly prevents leakage: it identifies the topic mismatch and ensures that generated snippets do not accidentally reveal the target information (Figure 12, center). This is the world-model equivalent of theory of mind: the model distinguishes what the *agent* knows from what the *environment* should reveal.

Case Study: Accurate Environment Prediction Enables Action Refinement

Task: mailman — Configure a Postfix + Mailman3 mailing list server so that emails to reading-group-join@local.edu are routed to Mailman’s LMTP service for subscription processing.

Shared Obstacle (both models encounter the same error):

```
550 5.1.1 <reading-group-join@local.edu>: Recipient address rejected: User unknown in local recipient table
```

Both models configure transport_maps to route list addresses to Mailman’s LMTP (lmtpl:[127.0.0.1]:8024). The transport lookup succeeds, yet Postfix still rejects the recipient. Resolving this requires an accurate world model of Postfix’s internal processing pipeline.

Before LWM RL

[Prediction] If I configure transport_maps to route to LMTP, Postfix will use that transport to deliver the mail. The recipient validation against local tables happens **after** transport routing decides the delivery path.
⇒ The transport config should be sufficient. ✗ **Incorrect**

[Action] Tries virtual_alias_maps, virtual_transport, relay configs...
Oscillates between transport_maps, virtual_maps, and relay configurations.
Never modifies local_recipient_maps. Task fails (timeout).

After LWM RL

[Prediction] local.edu is in mydestination. If I only configure transport_maps without modifying local_recipient_maps, Postfix will reject the recipient at the validation stage **before** transport routing is ever consulted.
⇒ The mail will never reach LMTP. ✓ **Correct**

[Refinement] The fix must target local_recipient_maps, not transport_maps.
→ Add Mailman addresses to the local recipient table so they pass validation before transport routes them to LMTP.

[Action] Create hash:/var/lib/mailman3/data/local_recipients with list addresses. ✓
All tests pass.

Figure 11: Case study of prediction-driven action refinement on the mailman task from Terminal-Bench 2.0.

Multi-Step Causal Reasoning. The model constructs causal chains that span multiple system abstractions. In one Terminal example, predicting the output of `curl -s localhost:3000 | python3 -m json.tool` requires a six-step chain: Node.js missing → server never started → no listener on port 3000 → curl fails silently (−s flag) → pipe receives empty input → json.tool raises a specific `JSONDecodeError`. Each step draws on different system knowledge (package management, process lifecycle, curl semantics, Python error messages), yet the model chains them correctly. This internalized causal simulation is the mechanism through which world-model training improves downstream agent performance: an agent with this knowledge can anticipate failures before executing actions.

7.2 RL Enhances Micro-Level Simulation Fidelity

Turn-level evaluation scores (§5.2) capture aggregate quality, but RL training also improves fidelity at a much finer granularity: individual URL identifiers, byte-level arithmetic, and cross-turn API schema consistency. Figure 13 illustrates these micro-level improvements.

Search: URL Realism across RL Steps. We trace a single Search-domain sample across RL checkpoints (Figure 13, top). At Step 100, the model generates an IMDB URL with a plausible but synthetic title identifier (tt2333444) and reasonable snippets. By Step 200, the identifier shifts to tt2988794, sources appear in natural ranking order (Wikipedia first, then IMDB, NYT, Rotten Tomatoes), and snippets contain query-specific factual detail that closely mirrors the ground truth. URL identifiers are a negligible fraction of total response tokens, yet RL enhances even these low-salience details, suggesting that the reward signal propagates below the granularity of explicit reward dimensions.

Terminal: Character-Level Byte Arithmetic. In a multi-turn trajectory, the model sees file content displayed by `cat` in an earlier turn, then must predict the output of `wc -c` several turns later. Rather than generating a plausible number, the model enumerates individual characters in its thinking trace,

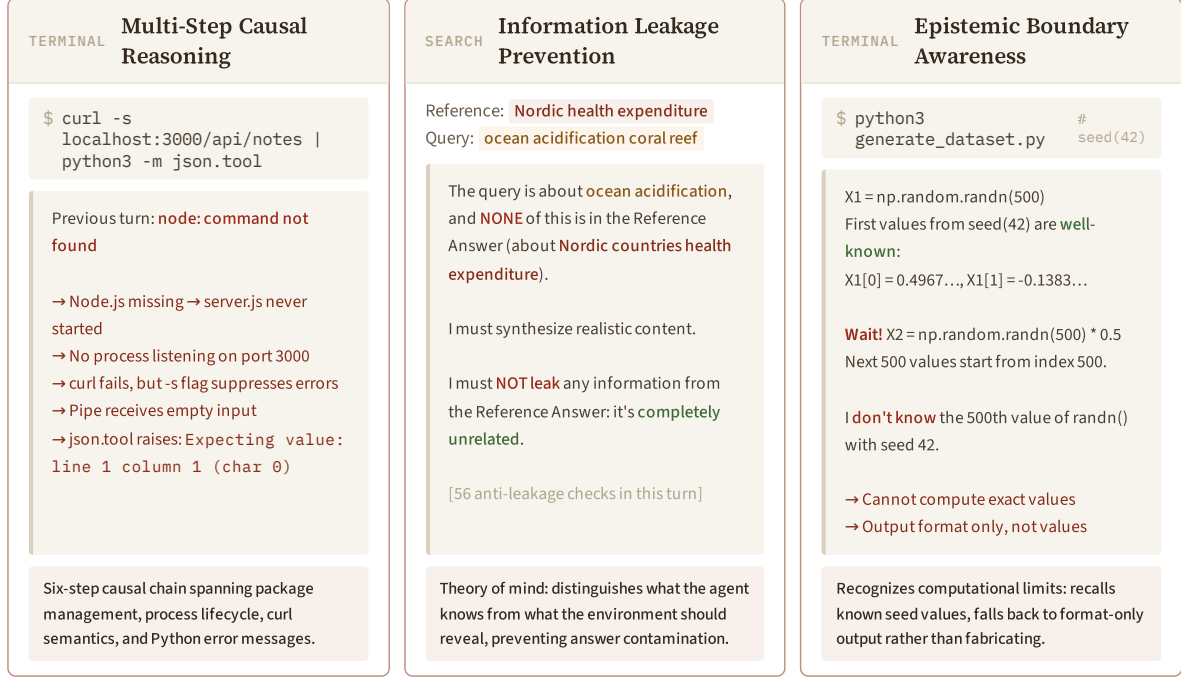


Figure 12: Representative LWM reasoning patterns from Qwen-AgentWorld-397B-A17B’s thinking traces. **Left:** Multi-step causal reasoning in Terminal, where a chain spans package management, process lifecycle, curl semantics, and Python errors. **Center:** Information leakage prevention in Search, where the model distinguishes what the agent knows from what the environment should reveal to prevent answer contamination. **Right:** Epistemic boundary awareness in Terminal, where the model recognizes computational limits and falls back to format-only output rather than fabricating unknowable values.

including invisible `\n` bytes, and arrives at the exact count (53 bytes) through letter-by-letter arithmetic (Figure 13, bottom left). In other examples, the model satisfies cryptographic invariants (identical files → identical SHA256 hashes) while simultaneously respecting Unix pipeline semantics (tee to a nonexistent directory fails without affecting stdout).

MCP: API Schema Fidelity. In an MCP trajectory simulating a Notion workspace (Figure 13, bottom right), the model maintains perfect consistency across nine sequential API calls: the same user identifier appears in every `created_by` field, each block’s `parent.page_id` matches the queried block ID, and the full Notion schema (~20 fields per block) is reproduced without omissions. The model implements a stateful in-context database, maintaining referential integrity across dozens of nested JSON objects.

8 Related Work

World Models. The concept of a world model as an internal simulator that predicts future states given actions is central to several frameworks for general intelligence (LeCun et al., 2022; Hafner et al., 2023; Yang, 2026; Delgrange, 2026). In visual and embodied domains, this idea has been instantiated through learned latent-space dynamics models. IRIS (Micheli et al., 2022) first demonstrates that Transformers can serve as sample-efficient world models for Atari games. DreamerV3 (Hafner et al., 2023) and its successor Dreamer 4 (Hafner et al., 2025) train agents entirely inside a learned world model, achieving strong performance across diverse control tasks without direct environment interaction during policy optimization. Video-based world models such as Cosmos (Ali et al., 2025), Genie 3 (Ball et al., 2025), and Vid2World (Huang et al., 2025a) scale this approach to high-resolution visual prediction, enabling interactive simulation of physical environments. GameNGen (Valevski et al., 2025) shows that diffusion models can simulate complex video games in real time. V-JEPA 2 (Assran et al., 2025) learns predictive representations from video without pixel-level reconstruction, demonstrating that self-supervised world models can support both understanding and planning. UniSim (Yang et al., 2023) learns a universal simulator of real-world interaction by orchestrating diverse visual datasets spanning objects, robotic actions, and navigation. Hou et al. (2026) provide a comprehensive survey of world models for robot learning, covering policy coupling and video world models across manipulation, navigation, and au-

SEARCH Simulation fidelity across RL steps — query: "Big Eyes 2014 film directed by Tim Burton"



Figure 13: Micro-level fidelity improvements during RL training. **Top:** Search domain: evolution of a single sample across RL steps. URL identifiers, source diversity, and snippet specificity all improve, despite occupying a tiny fraction of total output tokens. **Bottom left:** Terminal domain: the model performs exact byte-level arithmetic by enumerating characters including invisible newlines. **Bottom right:** MCP domain: the model maintains cross-turn schema consistency (user IDs, parent-child references, UUID formats) across nine Notion API calls.

tonomous driving. More broadly, these approaches model how the environment evolves over time, often conditioned on actions, and use the learned dynamics for planning or policy learning. Qwen-AgentWorld extends this paradigm to text-based agent environments, where the “observations” are structured text such as API responses, terminal outputs, accessibility trees, and UI view hierarchies.

Language World Models. A growing body of work explores whether LLMs can serve as world models for text-based agent environments (Li et al., 2026c). Li et al. (2025b) propose a three-level evaluation framework for LLM-based world models and find that fine-tuning substantially improves simulation fidelity, though gains depend on behavioral coverage and environment complexity; Wang et al. (2024) draw a complementary conclusion through systematic evaluation of LLMs as text-game simulators, showing that off-the-shelf LLMs remain unreliable world simulators. Richens et al. (2025) prove theoretically that any sufficiently general agent must contain a world model, and Cifuentes (2026) extend this result to partial observability and stochastic settings. On the empirical side, several concurrent works train LLMs as environment simulators. RLVR-World (Wu et al., 2026a) uses RL to train a unified world model that generates environment responses for agent training across text, web, and video domains, demonstrating that RL-trained simulators produce higher-fidelity predictions than SFT-only baselines. In the web domain, WebDreamer (Gu et al., 2024) and WMA (Chae et al., 2025) pioneer model-based planning with LLM-simulated web environments, WebWorld (Xiao et al., 2026) is the first open-web world model trained at scale advancing both agent training and inference-time search, and WebATLAS (Cheng et al., 2025) combines experience-driven memory with look-ahead action simulation. Shen et al. (2026b) augment web agents with world-model-based action correction, Imagine-then-Plan (Liu et al., 2026) uses

world-model lookahead for adaptive agent planning, DynaWeb (Ding et al., 2026) applies model-based RL to web agents, and WebSynthesis (Gao et al., 2025) combines world-model-guided MCTS with trajectory synthesis. Simia (Li et al., 2025c) uses reasoning models as environment simulators for agent training on the τ -bench task suite. RWML (Yu et al., 2026) formulates world-model learning as an RL problem and demonstrates that RL-trained world-modeling capability improves the model’s own agent performance. DyMo (Guo et al., 2025b) shows that language world modeling improves agent performance on interactive tasks, validating the benefit of next-state prediction as a training objective. Wang et al. (2025a) show that LLMs can serve as scalable, general-purpose simulators for digital agent training. Zhang et al. (2025) propose learning from *early experience*, where the agent uses its own interaction data with implicit world modeling as supervision, bypassing the need for external rewards. On the training methodology front, VAGEN (Wang et al., 2026a) reinforces world-model reasoning in VLM agents through dense state-prediction rewards, BehR (Huang et al., 2026) optimizes for behavior consistency with the real environment rather than surface-level state matching, Ren et al. (2026) align agentic world models via knowledgeable experience learning, and SWIRL (Qiu et al., 2026) self-improves world models from state-only sequences by alternating forward and inverse dynamics models without action labels. SSRL (Fan et al., 2025) and ZeroSearch (Sun et al., 2025) train LLMs via RL to simulate search engines, eliminating dependence on external APIs. ECHO (Shrivastava et al., 2026) shows that terminal agents can acquire world models by adding an auxiliary environment-prediction loss to standard RL, doubling pass rates on Terminal-Bench 2.0. In the GUI domain, Code2World (Zheng et al., 2026) generates renderable code as a proxy for GUI state prediction, while CUWM (Guan et al., 2026) builds a computer-use world model via two-stage factorization. MobileDreamer (Cao et al., 2026a) uses sketch-based GUI world models for mobile agent planning, Koh et al. (2026) propose generative visual code world models for mobile environments, and UISim (Xiang et al., 2025b) builds an interactive image-based UI simulator for dynamic mobile environments. SWE-World (Sun et al., 2026) proposes LLM-based environment simulation for software engineering agents, removing the dependence on Docker sandboxes, CWM (Copet et al., 2025) mid-trains a 32B open-weights LLM on observation-action trajectories from code interpreters and Docker environments to enable step-by-step simulation of code execution, and Rahmani (2026) analyze and debug failure modes in code world models. A complementary line uses LLMs to generate executable world models rather than serving as simulators directly: Code WM (Lehrach et al., 2025) translates game rules into Python simulators supporting MCTS planning, Text2World (Hu et al., 2025a) benchmarks LLMs on PDDL-based symbolic world model generation, and Agent2World (Hu et al., 2025b) proposes a multi-agent framework for the same task. WorldLLM (Levy et al., 2025) takes an orthogonal approach, enhancing world modeling through curiosity-driven theory-making without fine-tuning. Qian et al. (2026) provide an important negative result: current agents fail to leverage world models as tools for foresight-based planning, lacking the mechanisms to strategically invoke simulation and integrate its predictions. This finding motivates our native world-model approach, where environment modeling is the training objective from the CPT stage onward rather than bolted on after training.

Qwen-AgentWorld differs from these works in two respects. First, we develop a *native* language world model as a foundation model for agentic environment simulation: it covers 7 domains within a single model, and is trained through a three-stage CPT→SFT→RL pipeline that starts from environment modeling as the pre-training objective, rather than fine-tuning a general-purpose LLM post hoc. Second, we investigate how world modeling can improve general agents through two complementary paradigms: *decoupling* the agent from the world model, where turn-level controllability (partial observation, difficulty modulation) enables Sim RL that matches or exceeds Real RL; and *unifying* them into a single framework, where LWM pre-training serves as a warm-up that strengthens downstream agent performance.

Agent Foundation Models and Continual Pre-Training. AgentFounder (Su et al., 2025) proposes continual pre-training on agentic interaction data to build a 30B-parameter agent foundation model, demonstrating that domain-specific CPT improves downstream agent performance. TRUSTEE (Tang et al., 2026) shows that even a free 8B LLM can fully simulate tool-use environments, enabling zero-shot agent training without any real environment access. Chen et al. (2025) internalize world models via self-play fine-tuning. In the robotics domain, DreamZero (Ye et al., 2026b) shows that video-based world-action models trained on manipulation data can serve as zero-shot policies. Our work shares the CPT-stage insight with AgentFounder but differs in scope (seven domains vs. a smaller set) and in the subsequent SFT and RL stages that refine simulation fidelity beyond what CPT alone achieves.

Synthetic Environment Generation. A parallel line of work constructs synthetic environments programmatically rather than learning a neural simulator. AWM (Agent World Model) (Wang et al., 2026d) generates code-based environments for agentic RL, and Agent-World (Dong et al., 2026) scales real-world environment synthesis to nearly 2,000 environments with self-evolving training. RLAnything (Wang et al., 2026c) jointly forges environment, policy, and reward model in a fully dynamic RL system, and GenEnv (Guo et al., 2025a) co-evolves agents and environment simulators with difficulty-aligned curricula. ASTRA (Tian et al., 2026) automates the synthesis of agentic trajectories and reinforcement arenas

from tool-call graphs. Domain-specific generators include InfiniteWeb (Zhang et al., 2026), WebGym (Bai et al., 2026), Weblica (Kar et al., 2026), AutoWebWorld (Wu et al., 2026b), WebFactory (Fan et al., 2026), and Chae et al. (2026) for web environments; GUI-Genesis (Cao et al., 2026b), EvoCUA (Xue et al., 2026), and EE-MCP (He et al., 2026) for GUI and mobile environments; TermiGen (Zhu et al., 2026) and Endless Terminals (Gandhi et al., 2026) for terminal environments; and SWE-Universe (Chen et al., 2026), daVinci-Env (Fu et al., 2026), Zhao et al. (2026), SandMLE (Zhou et al., 2026), and ClawEnvKit (Li et al., 2026b) for software engineering and general agent platforms. ScaleEnv (Tu et al., 2026), AutoForge (Cai et al., 2025), EnvScaler (Song et al., 2026), SCALER (Xu et al., 2026a), and Fang et al. (2025) propose general frameworks for scaling environment synthesis. These code-driven approaches offer deterministic execution and verifiable rewards, but are inherently limited to domains where environments can be programmatically specified. Our LWM-based simulation is complementary: it trades determinism for generality, covering domains (e.g., search engines, real-world MCP servers) where code-based synthesis is impractical. Huang et al. (2025b) survey environment scaling methods for interactive agentic experience collection, and Chu et al. (2026) provide a broader treatment of agentic world modeling covering foundations, capabilities, and emerging directions.

9 Conclusion and Future Work

We presented Qwen-AgentWorld, the first family of native language world models covering seven agent interaction domains within a single model at two scales (35B-A3B and 397B-A17B). A three-stage recipe “CPT injects, SFT activates, RL sharpens” progressively injects environment knowledge, activates next-state-prediction reasoning, and sharpens simulation fidelity. We also introduced AgentWorldBench, a LWM benchmark that pairs every sample with a ground-truth observation from real environments. As a decoupled simulator, we validate the effectiveness of controllable simulation on 3 agentic benchmarks, surpassing both uncontrolled simulation and real-environment training. As a unified agent foundation model, LWM warm-up consistently improves downstream agent performance across 7 diverse tasks via cross-domain transfer, providing initial validation that LWMs can serve as a foundation for building stronger agent models. By enabling controllable simulation beyond real environments and establishing next-state prediction as a transferable agent foundation, language world modeling opens a new axis for scaling general agents beyond what real-environment interaction alone can provide.

Future Work.

- **Agent-LWM Co-Evolution.** Self-play where the agent discovers novel states that push the world model boundary, while the world model generates increasingly challenging scenarios for the agent.
- **Multimodal Extension.** Fusing GUI screenshots with text-based state representations to unify visual and language world models for Android, Web, and OS domains.
- **Adaptive Sim-to-Real Routing.** Learning a router that decides per query whether to invoke the world model or the real environment, balancing cost against fidelity.
- **Dynamic Tool Synthesis.** Using the world model to synthesize new tools on the fly rather than relying on a predefined tool set.

10 Authors

Core Contributors: Yuxin Zuo, Zikai Xiao, Li Sheng, Fei Huang[†], Jianhong Tu[†], Yuxuan Liu, Tianyi Tang, Xiaomeng Hu, Yang Su, Qingfeng Lan, Yantao Liu, Qin Zhu, Yinger Zhang, Bowen Yu, An Yang, Dayiheng Liu, Jingren Zhou

Contributors (ordered alphabetically): Haiquan Zhao, Haiyang Xu, Jianxin Yang, Jiayang Cheng, Junyang Wang, Lianghao Deng, Mingfeng Xue, Tianyi Bai, Yang Fan, Yubo Ma, Yucheng Li, Zeyu Cui, Zhihai Wang, Zhihui Xie, Zhuorui Ye

External Advisor: Ning Ding (Tsinghua University)

[†]Project Lead.

References

Arslan Ali, Junjie Bai, Maciej Bala, Yogesh Balaji, Aaron Blakeman, Tiffany Cai, Jiaxin Cao, Tianshi Cao, Elizabeth Cha, Yu-Wei Chao, et al. World simulation with video foundation models for physical ai. *arXiv preprint arXiv:2511.00062*, 2025.

-
- Anthropic. System card: Claude sonnet 4.5, 2025. URL <https://assets.anthropic.com/m/12f214efcc2f457a/original/Claude-Sonnet-4-5-System-Card.pdf>.
- Anthropic. System card: Claude opus 4.6, 2026a. URL <https://www-cdn.anthropic.com/14e4fb01875d2a69f646fa5e574dea2b1c0ff7b5.pdf>.
- Anthropic. System card: Claude opus 4.8, 2026b. URL <https://www-cdn.anthropic.com/0f0c97ad20d8005706296bd92aa1c27c6b2f4f61.pdf>.
- Anthropic. System card: Claude sonnet 4.6, 2026c. URL <https://www-cdn.anthropic.com/bbd8ef16d70b7a1665f14f306ee88b53f686aa75.pdf>.
- Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zhohus, et al. V-jepa 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.
- Hao Bai, Alexey Taymanov, Tong Zhang, Aviral Kumar, and Spencer Whitehead. Webgym: Scaling training environments for visual web agents with realistic tasks. *arXiv preprint arXiv:2601.02439*, 2026.
- Philip J. Ball, Jakob Bauer, Frank Belletti, Bethanie Brownfield, Ariel Ephrat, Shlomi Fruchter, Agrim Gupta, Kristian Holsheimer, Aleksander Holynski, Jiri Hron, Christos Kaplanis, Marjorie Limont, Matt McGill, Yanko Oliveira, Jack Parker-Holder, Frank Perbet, Guy Scully, Jeremy Shar, Stephen Spencer, Omer Tov, Ruben Villegas, Emma Wang, Jessica Yung, Cip Baetu, Jordi Berbel, David Bridson, Jake Bruce, Gavin Buttimore, Sarah Chakera, Bilva Chandra, Paul Collins, Alex Cullum, Bogdan Damoc, Vibha Dasagi, Maxime Gazeau, Charles Gbadamosi, Woohyun Han, Ed Hirst, Ashyana Kachra, Lucie Kerley, Kristian Kjems, Eva Knoepfel, Vika Koriakin, Jessica Lo, Cong Lu, Zeb Mehring, Alex Moufarek, Henna Nandwani, Valeria Oliveira, Fabio Pardo, Jane Park, Andrew Pierson, Ben Poole, Helen Ran, Tim Salimans, Manuel Sanchez, Igor Saprykin, Amy Shen, Sailesh Sidhwani, Duncan Smith, Joe Stanton, Hamish Tomlinson, Dimple Vijaykumar, Luyu Wang, Piers Wingfield, Nat Wong, Keyang Xu, Christopher Yew, Nick Young, Vadim Zubov, Douglas Eck, Dumitru Erhan, Koray Kavukcuoglu, Demis Hassabis, Zoubin Ghahramani, Raia Hadsell, Aaron van den Oord, Inbar Mosseri, Adrian Bolton, Satinder Singh, and Tim Rocktäschel. Genie 3: A new frontier for world models, 2025. URL <https://deepmind.google/blog/genie-3-a-new-frontier-for-world-models/>.
- Shihao Cai, Runnan Fang, Jialong Wu, Baixuan Li, Xinyu Wang, Yong Jiang, Liangcai Su, Liwen Zhang, Wenbiao Yin, Zhen Zhang, et al. Autoforge: Automated environment synthesis for agentic reinforcement learning. *arXiv preprint arXiv:2512.22857*, 2025.
- Yilin Cao, Yufeng Zhong, Zhixiong Zeng, Liming Zheng, Jing Huang, Haibo Qiu, Peng Shi, Wenji Mao, and Wan Guanglu. Mabledreamer: Generative sketch world model for gui agent. *arXiv preprint arXiv:2601.04035*, 2026a.
- Yuan Cao, Dezhi Ran, Mengzhou Wu, Yuzhe Guo, Xin Chen, Ang Li, Gang Cao, Gong Zhi, Hao Yu, Linyi Li, et al. Gui-genesis: Automated synthesis of efficient environments with verifiable rewards for gui agent post-training. *arXiv preprint arXiv:2602.14093*, 2026b.
- Hyunjoo Chae, Namyoung Kim, Kai Ong, Minju Gwak, Gwanwoo Song, Jihoon Kim, Sunghwan Kim, Dongha Lee, and Jinyoung Yeo. Web agents with world models: Learning and leveraging environment dynamics in web navigation. In *International Conference on Learning Representations*, volume 2025, pp. 63707–63738, 2025.
- Hyunjoo Chae, Jungsoo Park, and Alan Ritter. Safe and scalable web agent learning via recreated websites. *arXiv preprint arXiv:2603.10505*, 2026.
- Mouxian Chen, Lei Zhang, Yunlong Feng, Xuwu Wang, Wenting Zhao, Ruisheng Cao, Jiayi Yang, Jiawei Chen, Mingze Li, Zeyao Ma, et al. Swe-universe: Scale real-world verifiable environments to millions. *arXiv preprint arXiv:2602.02361*, 2026.
- Shiqi Chen, Tongyao Zhu, Zian Wang, Jinghan Zhang, Kangrui Wang, Siyang Gao, Teng Xiao, Yee Whye Teh, Junxian He, and Manling Li. Internalizing world models via self-play finetuning for agentic rl. *arXiv preprint arXiv:2510.15047*, 2025.
- Jiali Cheng, Anjishnu Kumar, Roshan Lal, Rishi Rajasekaran, Hani Ramezani, Omar Zia Khan, Oleg Rokhlenko, Sunny Chiu-Webster, Gang Hua, and Hadi Amiri. Webatlas: An llm agent with experience-driven memory and action simulation. *arXiv preprint arXiv:2510.22732*, 2025.
- Meng Chu, Xuan Billy Zhang, Kevin Qinghong Lin, Lingdong Kong, Jize Zhang, Teng Tu, Weijian Ma, Ziqi Huang, Senqiao Yang, Wei Huang, et al. Agentic world modeling: Foundations, capabilities, laws, and beyond. *arXiv preprint arXiv:2604.22748*, 2026.

-
- Santiago Cifuentes. General agents contain world models, even under partial observability and stochasticity. *arXiv preprint arXiv:2602.03146*, 2026.
- Jade Copet, Quentin Carbonneaux, Gal Cohen, Jonas Gehring, Jacob Kahn, Jannik Kossen, Felix Kreuk, Emily McMilin, Michel Meyer, Yuxiang Wei, et al. Cwm: An open-weights llm for research on code generation with world models. *arXiv preprint arXiv:2510.02387*, 2025.
- Google DeepMind. Gemini 3 flash model card, 2025. URL <https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-3-Flash-Model-Card.pdf>.
- Google DeepMind. Gemini 3.1 pro model card, 2026. URL <https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-3-1-Pro-Model-Card.pdf>.
- DeepSeek-AI. Deepseek-v4: Towards highly efficient million-token context intelligence, 2026.
- Florent Delgrange. Foundation world models for agents that learn, verify, and adapt reliably beyond static environments. *arXiv preprint arXiv:2602.23997*, 2026.
- Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, et al. Swe-bench pro: Can ai agents solve long-horizon software engineering tasks? *arXiv preprint arXiv:2509.16941*, 2025.
- Hang Ding, Peidong Liu, Junqiao Wang, Ziwei Ji, Meng Cao, Rongzhao Zhang, Lynn Ai, Eric Yang, Tianyu Shi, and Lei Yu. Dynaweb: Model-based reinforcement learning of web agents. *arXiv preprint arXiv:2601.22149*, 2026.
- Guanting Dong, Junting Lu, Junjie Huang, Wanjuan Zhong, Longxiang Liu, Shijue Huang, Zhenyu Li, Yang Zhao, Xiaoshuai Song, Xiaoxi Li, et al. Agent-world: Scaling real-world environment synthesis for evolving general agent intelligence. *arXiv preprint arXiv:2604.18292*, 2026.
- Sicheng Fan, Qingyun Shi, Shengze Xu, Shengbo Cai, Tiejong Zeng, Li Ling, Yanyi Shang, and Dehan Kong. Webfactory: Automated compression of foundational language intelligence into grounded web agents. *arXiv preprint arXiv:2603.05044*, 2026.
- Yuchen Fan, Kaiyan Zhang, Heng Zhou, Yuxin Zuo, Yanxu Chen, Yu Fu, Xinwei Long, Xuekai Zhu, Che Jiang, Yuchen Zhang, et al. Ssr: Self-search reinforcement learning. *arXiv preprint arXiv:2508.10874*, 2025.
- Runnan Fang, Shihao Cai, Baixuan Li, Jialong Wu, Guangyu Li, Wenbiao Yin, Xinyu Wang, Xiaobin Wang, Liangcai Su, Zhen Zhang, et al. Towards general agentic intelligence via environment scaling. *arXiv preprint arXiv:2509.13311*, 2025.
- Dayuan Fu, Shenyu Wu, Yunze Wu, Zerui Peng, Yaxing Huang, Jie Sun, Ji Zeng, Mohan Jiang, Lin Zhang, Yukun Li, et al. davinci-env: Open swe environment synthesis at scale. *arXiv preprint arXiv:2603.13023*, 2026.
- Kanishk Gandhi, Shivam Garg, Noah D Goodman, and Dimitris Papailiopoulos. Endless terminals: Scaling rl environments for terminal agents. *arXiv preprint arXiv:2601.16443*, 2026.
- Yifei Gao, Junhong Ye, Jiaqi Wang, and Jitao Sang. Websynthesis: World-model-guided mcts for efficient webui-trajectory synthesis. *arXiv preprint arXiv:2507.04370*, 2025.
- Yu Gu, Kai Zhang, Yuting Ning, Boyuan Zheng, Boyu Gou, Tianci Xue, Cheng Chang, Sanjari Srivastava, Yanan Xie, Peng Qi, et al. Is your llm secretly a world model of the internet? model-based planning for web agents. *arXiv preprint arXiv:2411.06559*, 2024.
- Yiming Guan, Rui Yu, John Zhang, Lu Wang, Chaoyun Zhang, Liqun Li, Bo Qiao, Si Qin, He Huang, Fangkai Yang, et al. Computer-using world model. *arXiv preprint arXiv:2602.17365*, 2026.
- Anisha Gunjal, Anthony Wang, Elaine Lau, Vaskar Nath, Yunzhong He, Bing Liu, and Sean Hendryx. Rubrics as rewards: Reinforcement learning beyond verifiable domains. *arXiv preprint arXiv:2507.17746*, 2025.
- Jiacheng Guo, Ling Yang, Peter Chen, Qixin Xiao, Yinjie Wang, Xinzhe Juan, Jiahao Qiu, Ke Shen, and Mengdi Wang. Genenv: Difficulty-aligned co-evolution between llm agents and environment simulators. *arXiv preprint arXiv:2512.19682*, 2025a.
- Shangmin Guo, Omar Darwiche Domingues, Raphaël Avalos, Aaron Courville, and Florian Strub. World modelling improves language model agents. *arXiv preprint arXiv:2506.02918*, 2025b.

-
- Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.
- Danijar Hafner, Wilson Yan, and Timothy Lillicrap. Training agents inside of scalable world models. *arXiv preprint arXiv:2509.24527*, 2025.
- Tiantian He, Yihang Chen, Keyue Jiang, Ka Yiu Lee, Kaiwen Zhou, Kun Shao, and Shuai Wang. Ee-mcp: Self-evolving mcp-gui agents via automated environment generation and experience learning. *arXiv preprint arXiv:2604.09815*, 2026.
- Bohan Hou, Gen Li, Jindou Jia, Tuo An, Xinying Guo, Sicong Leng, Haoran Geng, Yanjie Ze, Tatsuya Harada, Philip Torr, et al. World model for robot learning: A comprehensive survey. *arXiv preprint arXiv:2605.00080*, 2026.
- Mengkang Hu, Tianxing Chen, Yude Zou, Yuheng Lei, Qiguang Chen, Ming Li, Yao Mu, Hongyuan Zhang, Wenqi Shao, and Ping Luo. Text2world: Benchmarking large language models for symbolic world model generation. In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 26043–26066, 2025a.
- Mengkang Hu, Bowei Xia, Yuran Wu, Ailing Yu, Yude Zou, Qiguang Chen, Shijian Wang, Jiarui Jin, Kexin Li, Wenxiang Jiao, et al. Agent2world: Learning to generate symbolic world models via adaptive multi-agent feedback. *arXiv preprint arXiv:2512.22336*, 2025b.
- Siqiao Huang, Jialong Wu, Qixing Zhou, Shangchen Miao, and Mingsheng Long. Vid2world: Crafting video diffusion models to interactive world models. *arXiv preprint arXiv:2505.14357*, 2025a.
- Youling Huang, Guanqiao Chen, Junchi Yao, Lu Wang, Fangkai Yang, Chao Du, ChenZhuo Zhao, Pu Zhao, Qingwei Lin, Saravan Rajmohan, et al. Beyond state consistency: Behavior consistency in text-based world models. *arXiv preprint arXiv:2604.13824*, 2026.
- Yuchen Huang, Sijia Li, Minghao Liu, Wei Liu, Shijue Huang, Zhiyuan Fan, Hou Pong Chan, and Yi R Fung. Environment scaling for interactive agentic experience collection: A survey. *arXiv preprint arXiv:2511.09586*, 2025b.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, volume 2024, pp. 54107–54157, 2024.
- Oğuzhan Fatih Kar, Roman Bachmann, Yuanzheng Gong, Anders Boesen Lindbo Larsen, and Afshin Dehghan. Weblica: Scalable and reproducible training environments for visual web agents, 2026. URL <https://arxiv.org/abs/2605.06761>.
- Andrej Karpathy. autoresearch. <https://github.com/karpathy/autoresearch>, 2026.
- Woosung Koh, Sungjun Han, Segyu Lee, Se-Young Yun, and Jamin Shin. Generative visual code mobile world models. *arXiv preprint arXiv:2602.01576*, 2026.
- Yann LeCun et al. A path towards autonomous machine intelligence version 0.9. 2, 2022-06-27. *Open Review*, 62(1):1–62, 2022.
- Wolfgang Lehrach, Daniel Hennes, Miguel Lazaro-Gredilla, Xinghua Lou, Carter Wendelken, Zun Li, Antoine Dedieu, Jordi Grau-Moya, Marc Lanctot, Atil Iscen, et al. Code world models for general game playing. *arXiv preprint arXiv:2510.04542*, 2025.
- Guillaume Levy, Cédric Colas, Pierre-Yves Oudeyer, Thomas Carta, and Clément Romac. Worldllm: Improving llms’ world modeling using curiosity-driven theory-making. *arXiv preprint arXiv:2506.06725*, 2025.
- Junlong Li, Wenshuo Zhao, Jian Zhao, Weihao Zeng, Haoze Wu, Xiaochen Wang, Rui Ge, Yuxuan Cao, Yuzhen Huang, Wei Liu, et al. The tool decathlon: Benchmarking language agents for diverse, realistic, and long-horizon task execution. *arXiv preprint arXiv:2510.25726*, 2025a.
- Wanli Li, Bince Qu, Bo Pan, Jianyu Zhang, Zheng Liu, Pan Zhang, Wei Chen, and Bo Zhang. Literesearcher: A scalable agentic rl training framework for deep research agent. *arXiv preprint arXiv:2604.17931*, 2026a.
- Xirui Li, Ming Li, Ion Stoica, Cho-Jui Hsieh, and Tianyi Zhou. Clawenvkit: Automatic environment generation for claw-like agents. *arXiv preprint arXiv:2604.18543*, 2026b.

-
- Yixia Li, Hongru Wang, Jiahao Qiu, Zhenfei Yin, Dongdong Zhang, Cheng Qian, Zeping Li, Pony Ma, Guanhua Chen, and Heng Ji. From word to world: Can large language models be implicit text-based world models? *arXiv preprint arXiv:2512.18832*, 2025b.
- Yixia Li, Hongru Wang, Peng Lai, Zhiwen Ruan, He Zhu, Youxin Zhu, Ganlong Zhao, Minda Hu, Yun Chen, Sibe Yang, Peng Li, Jeff Z. Pan, Jia Pan, Guanhua Chen, Yang Liu, and Guanbin Li. Bridging the agent-world gap: Text world models for llm-based agents, 2026c. URL <https://arxiv.org/abs/2606.09032>.
- Yuetai Li, Huseyin A Inan, Xiang Yue, Wei-Ning Chen, Lukas Wutschitz, Janardhan Kulkarni, Radha Poovendran, Robert Sim, and Saravan Rajmohan. Simulating environments with reasoning models for agent training. *arXiv preprint arXiv:2511.01824*, 2025c.
- Youwei Liu, Jian Wang, Hanlin Wang, Beichen Guo, and Wenjie Li. Imagine-then-plan: Agent learning from adaptive lookahead with world models. *arXiv preprint arXiv:2601.08955*, 2026.
- Mike A Merrill, Alexander G Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E Kelly Buchanan, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *arXiv preprint arXiv:2601.11868*, 2026.
- Vincent Micheli, Eloi Alonso, and François Fleuret. Transformers are sample-efficient world models. *arXiv preprint arXiv:2209.00588*, 2022.
- AI MiniMax. Minimax m2. 7: Early echoes of self-evolution, 2026.
- OpenAI. Introducing gpt-5.2, 2025. URL <https://openai.com/index/introducing-gpt-5-2/>.
- OpenAI. Introducing gpt-5.4, 2026. URL <https://openai.com/index/introducing-gpt-5-4/>.
- OpenClaw. Openclaw. <https://github.com/openclaw/openclaw>, 2026. Open-source personal AI assistant, version 2026.3.8, accessed 2026-03-09.
- Shishir G Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E Gonzalez. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *Forty-second International Conference on Machine Learning*, 2025.
- Tejal Patwardhan, Rachel Dias, Elizabeth Proehl, Grace Kim, Michele Wang, Olivia Watkins, Simón Posada Fishman, Marwan Aljubei, Phoebe Thacker, Laurance Fauconnet, et al. Gdpval: Evaluating ai model performance on real-world economically valuable tasks. *arXiv preprint arXiv:2510.04374*, 2025.
- Cheng Qian, Emre Can Acikgoz, Bingxuan Li, Xiusi Chen, Yuji Zhang, Bingxiang He, Qinyu Luo, Dilek Hakkani-Tür, Gokhan Tur, Yunzhu Li, et al. Current agents fail to leverage world model as tool for foresight. *arXiv preprint arXiv:2601.03905*, 2026.
- Yifu Qiu, Zheng Zhao, Waylon Li, Yftah Ziser, Anna Korhonen, Shay B Cohen, and Edoardo M Ponti. Self-improving world modelling with latent actions. *arXiv preprint arXiv:2602.06130*, 2026.
- Qwen Team. Qwen3.6-35B-A3B: Agentic coding power, now open to all, April 2026a. URL <https://qwen.ai/blog?id=qwen3.6-35b-a3b>.
- Qwen Team. Qwen3.6-Max-Preview: Smarter, sharper, still evolving, April 2026b. URL <https://qwen.ai/blog?id=qwen3.6-max-preview>.
- Qwen Team. Qwen3.6-Plus: Towards real world agents, April 2026c. URL <https://qwen.ai/blog?id=qwen3.6>.
- Babak Rahmani. Debugging code world models. *arXiv preprint arXiv:2602.07672*, 2026.
- Baochang Ren, Yunzhi Yao, Rui Sun, Shuofei Qiao, Ningyu Zhang, and Huajun Chen. Aligning agentic world models via knowledgeable experience learning. *arXiv preprint arXiv:2601.13247*, 2026.
- Jonathan Richens, David Abel, Alexis Bellot, and Tom Everitt. General agents contain world models. *arXiv preprint arXiv:2506.01622*, 2025.
- William F Shen, Xinchu Qiu, Chenxi Whitehouse, Lisa Alazraki, Shashwat Goel, Francesco Barbieri, Timon Willi, Akhil Mathur, and Ilias Leontiadis. Rethinking rubric generation for improving llm judge and reward modeling for open-ended tasks. *arXiv preprint arXiv:2602.05125*, 2026a.
- Zhouzhou Shen, Xueyu Hu, Xiyun Li, Tianqing Fang, Juncheng Li, and Shengyu Zhang. World-model-augmented web agents with action correction. *arXiv preprint arXiv:2602.15384*, 2026b.

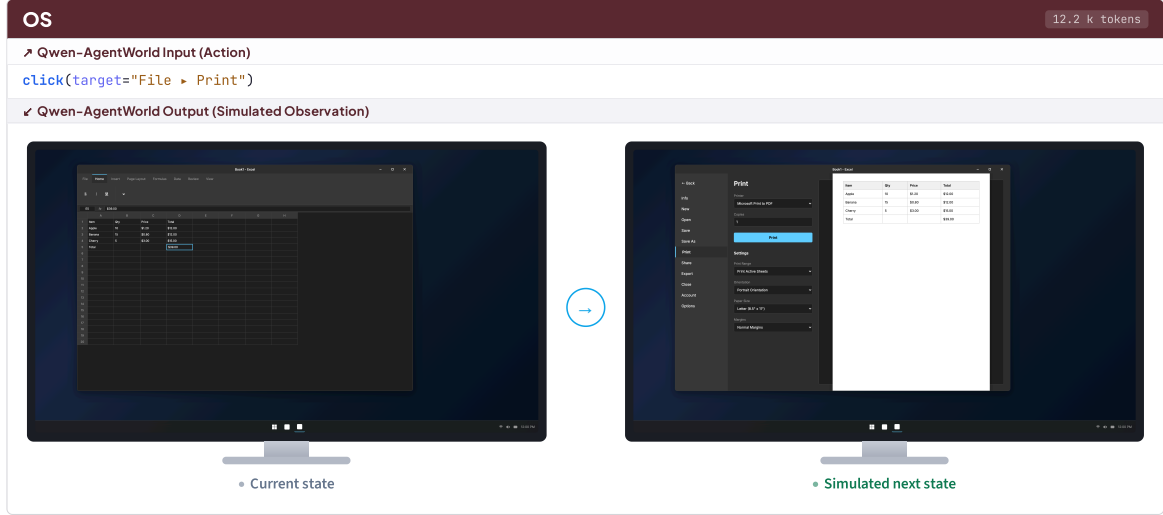
-
- Vaishnavi Shrivastava, Piero Kauffmann, Ahmed Awadallah, and Dimitris Papailiopoulos. Echo: Terminal agents learn world models for free. *arXiv preprint arXiv:2605.24517*, 2026.
- Xiaoshuai Song, Haofei Chang, Guanting Dong, Yutao Zhu, Ji-Rong Wen, and Zhicheng Dou. Envs-calor: Scaling tool-interactive environments for llm agent via programmatic synthesis. *arXiv preprint arXiv:2601.05808*, 2026.
- Liangcai Su, Zhen Zhang, Guangyu Li, Zhuo Chen, Chenxi Wang, Maojia Song, Xinyu Wang, Kuan Li, Jialong Wu, Xuanzhong Chen, et al. Scaling agents via continual pre-training. *arXiv preprint arXiv:2509.13310*, 2025.
- Hao Sun, Zile Qiao, Jiayan Guo, Xuanbo Fan, Yingyan Hou, Yong Jiang, Pengjun Xie, Yan Zhang, Fei Huang, and Jingren Zhou. Zerosearch: Incentivize the search capability of llms without searching. *arXiv preprint arXiv:2505.04588*, 2025.
- Shuang Sun, Huatong Song, Lisheng Huang, Jinhao Jiang, Ran Le, Zhihao Lv, Zongchao Chen, Yiwen Hu, Wenyang Luo, Wayne Xin Zhao, et al. Swe-world: Building software engineering agents in docker-free environments. *arXiv preprint arXiv:2602.03419*, 2026.
- Chenming Tang, Hsiu-Yuan Huang, Weijie Liu, Junqiang Zheng, Saiyong Yang, and Yunfang Wu. Democratizing tool learning with environments fully simulated by a free 8b language model, 2026. URL <https://arxiv.org/abs/2604.17739>.
- Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, SH Cai, Yuan Cao, Y Charles, HS Che, Cheng Chen, Guanduo Chen, et al. Kimi k2. 5: Visual agentic intelligence. *arXiv preprint arXiv:2602.02276*, 2026.
- Qwen Team. Qwen3.5: Accelerating productivity with native multimodal agents, February 2026. URL <https://qwen.ai/blog?id=qwen3.5>.
- Qwen Team and Alibaba Data. QwenClawBench: Real-user-distribution benchmark for openclaw agents, 2026. URL github.com/SKYLENAGE-AI/QwenClawBench.
- Xiaoyu Tian, Haotian Wang, Shuaiting Chen, Hao Zhou, Kaichi Yu, Yudian Zhang, Jade Ouyang, Junxi Yin, Jiong Chen, Baoyan Guo, et al. Astra: Automated synthesis of agentic trajectories and reinforcement arenas. *arXiv preprint arXiv:2601.21558*, 2026.
- Dunwei Tu, Hongyan Hao, Hansi Yang, Yihao Chen, Yi-Kai Zhang, Zhikang Xia, Yu Yang, Yueqing Sun, Xingchen Liu, Furao Shen, et al. Scaleenv: Scaling environment synthesis from scratch for generalist interactive tool-use agent training. *arXiv preprint arXiv:2602.06820*, 2026.
- Dani Valevski, Yaniv Leviathan, Moab Arar, and Shlomi Fruchter. Diffusion models are real-time game engines. In *International Conference on Learning Representations*, volume 2025, pp. 73754–73776, 2025.
- Kangrui Wang, Pingyue Zhang, Zihan Wang, Yaning Gao, Linjie Li, Qineng Wang, Hanyang Chen, Yiping Lu, Zhengyuan Yang, Lijuan Wang, et al. Vagen: Reinforcing world model reasoning for multi-turn vlm agents. *Advances in Neural Information Processing Systems*, 38:172871–172933, 2026a.
- Ruoyao Wang, Graham Todd, Ziang Xiao, Xingdi Yuan, Marc-Alexandre Côté, Peter Clark, and Peter Jansen. Can language models serve as text-based world simulators? In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pp. 1–17, 2024.
- Xiaohua Wang, Muzhao Tian, Yuqi Zeng, Zisu Huang, Jiakang Yuan, Bowen Chen, Jingwen Xu, Mingbo Zhou, Wenhao Liu, Muling Wu, et al. Reward hacking in the era of large models: Mechanisms, emergent misalignment, challenges. *arXiv preprint arXiv:2604.13602*, 2026b.
- Yiming Wang, Da Yin, Yuedong Cui, Ruichen Zheng, Zhiqian Li, Zongyu Lin, Di Wu, Xueqing Wu, Chenchen Ye, Yu Zhou, et al. Llms as scalable, general-purpose simulators for evolving digital agent training. *arXiv preprint arXiv:2510.14969*, 2025a.
- Yinjie Wang, Tianbao Xie, Ke Shen, Mengdi Wang, and Ling Yang. Rlanything: Forge environment, policy, and reward model in completely dynamic rl system. *arXiv preprint arXiv:2602.02488*, 2026c.
- Zhaoyang Wang, Canwen Xu, Boyi Liu, Yite Wang, Siwei Han, Zhewei Yao, Huaxiu Yao, and Yuxiong He. Agent world model: Infinity synthetic environments for agentic reinforcement learning. *arXiv preprint arXiv:2602.10090*, 2026d.
- Zihan Wang, Kangrui Wang, Qineng Wang, Pingyue Zhang, Linjie Li, Zhengyuan Yang, Xing Jin, Kefan Yu, Minh Nhat Nguyen, Licheng Liu, et al. Ragen: Understanding self-evolution in llm agents via multi-turn reinforcement learning. *arXiv preprint arXiv:2504.20073*, 2025b.

-
- Ryan Wong, Jiawei Wang, Junjie Zhao, Li Chen, Yan Gao, Long Zhang, Xuan Zhou, Zuo Wang, Kai Xiang, Ge Zhang, et al. Widesearch: Benchmarking agentic broad info-seeking. *arXiv preprint arXiv:2508.07999*, 2025.
- World Labs team. Marble: A multimodal world model. World Labs Technical Post, 2025. URL <https://www.worldlabs.ai/blog/marble-world-model>.
- Jialong Wu, Shaofeng Yin, Ningya Feng, and Mingsheng Long. Rlv-r-world: Training world models with reinforcement learning. *Advances in Neural Information Processing Systems*, 38:125312–125350, 2026a.
- Yifan Wu, Yiran Peng, Yiyu Chen, Jianhao Ruan, Zijie Zhuang, Cheng Yang, Jiayi Zhang, Man Chen, Yenchu Tseng, Zhaoyang Yu, et al. Autowebworld: Synthesizing infinite verifiable web environments via finite state machines. *arXiv preprint arXiv:2602.14296*, 2026b.
- Zijian Wu, Xiangyan Liu, Xinyuan Zhang, Lingjun Chen, Fanqing Meng, Lingxiao Du, Yiran Zhao, Fanshi Zhang, Yaoqi Ye, Jiawei Wang, et al. Mcpmark: A benchmark for stress-testing realistic and comprehensive mcp use. *arXiv preprint arXiv:2509.24002*, 2025.
- Jiannan Xiang, Yi Gu, Zihan Liu, Zeyu Feng, Qiyue Gao, Yiyan Hu, Benhao Huang, Guangyi Liu, Yichi Yang, Kun Zhou, et al. Pan: A world model for general, interactable, and long-horizon world simulation. *arXiv preprint arXiv:2511.09057*, 2025a.
- Jiannan Xiang, Yun Zhu, Lei Shu, Maria Wang, Lijun Yu, Gabriel Barcik, James Lyon, Srinivas Sunkara, and Jindong Chen. Uisim: An interactive image-based ui simulator for dynamic mobile environments. *arXiv preprint arXiv:2509.21733*, 2025b.
- Zikai Xiao, Jianhong Tu, Chuhan Zou, Yuxin Zuo, Zhi Li, Peng Wang, Bowen Yu, Fei Huang, Junyang Lin, and Zuoqiu Liu. Webworld: A large-scale world model for web agent training. *arXiv preprint arXiv:2602.14721*, 2026.
- Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh J Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *Advances in Neural Information Processing Systems*, 37: 52040–52094, 2024.
- Caijun Xu, Changyi Xiao, Zhongyuan Peng, Xinrun Wang, and Yixin Cao. Scaler: Synthetic scalable adaptive learning environment for reasoning. *arXiv preprint arXiv:2601.04809*, 2026a.
- Siyuan Xu, Shiyang Li, Xin Liu, Tianyi Liu, Yixiao Li, Zhan Shi, Zixuan Zhang, Zilong Wang, Qingyu Yin, Jianshu Chen, et al. Controllable and verifiable tool-use data synthesis for agentic reinforcement learning. *arXiv preprint arXiv:2604.09813*, 2026b.
- Taofeng Xue, Chong Peng, Mianqiu Huang, Linsen Guo, Tiancheng Han, Haozhe Wang, Jianing Wang, Xiaocheng Zhang, Xin Yang, Dengchang Zhao, et al. Evocua: Evolving computer use agents via learning from scalable synthetic experience. *arXiv preprint arXiv:2601.15876*, 2026.
- Sherry Yang. World models as an intermediary between agents and the real world. *arXiv preprint arXiv:2602.00785*, 2026.
- Sherry Yang, Yilun Du, Kamyar Ghasemipour, Jonathan Tompson, Leslie Kaelbling, Dale Schuurmans, and Pieter Abbeel. Learning interactive real-world simulators. *arXiv preprint arXiv:2310.06114*, 2023.
- Bowen Ye, Rang Li, Qibin Yang, Yuanxin Liu, Linli Yao, Hanglong Lv, Zhihui Xie, Chenxin An, Lei Li, Lingpeng Kong, et al. Claw-eval: Toward trustworthy evaluation of autonomous agents. *arXiv preprint arXiv:2604.06132*, 2026a.
- Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyan Gao, Sihyun Yu, George Kurian, Suneel Indupuru, You Liang Tan, Chunling Zhu, Jiannan Xiang, et al. World action models are zero-shot policies. *arXiv preprint arXiv:2602.15922*, 2026b.
- Xiao Yu, Baolin Peng, Ruize Xu, Yelong Shen, Pengcheng He, Suman Nath, Nikhil Singh, Jiangfeng Gao, and Zhou Yu. Reinforcement world model learning for llm-based agents. *arXiv preprint arXiv:2602.05842*, 2026.
- Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, Da Yin, Chendi Ge, Chenghua Huang, Chengxing Xie, et al. Glm-5: from vibe coding to agentic engineering. *arXiv preprint arXiv:2602.15763*, 2026.

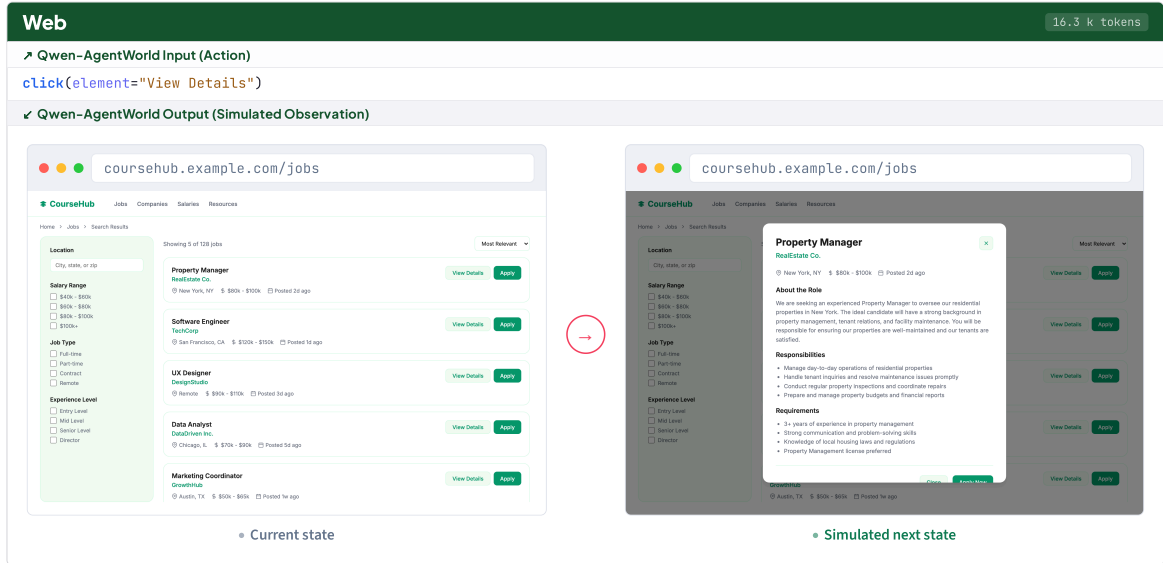
-
- Kai Zhang, Xiangchao Chen, Bo Liu, Tianci Xue, Zeyi Liao, Zhihan Liu, Xiyao Wang, Yuting Ning, Zhaorun Chen, Xiaohan Fu, et al. Agent learning via early experience. *arXiv preprint arXiv:2510.08558*, 2025.
- Ziyun Zhang, Zezhou Wang, Xiaoyi Zhang, Zongyu Guo, Jiahao Li, Bin Li, and Yan Lu. Infiniteweb: Scalable web environment synthesis for gui agent training. *arXiv preprint arXiv:2601.04126*, 2026.
- Jiale Zhao, Guoxin Chen, Fanzhe Meng, Minghao Li, Jie Chen, Hui Xu, Yongshuai Sun, Wayne Xin Zhao, Ruihua Song, Yuan Zhang, et al. Immersion in the github universe: Scaling coding agents to mastery. *arXiv preprint arXiv:2602.09892*, 2026.
- Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, et al. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025.
- Yuhao Zheng, Li'an Zhong, Yi Wang, Rui Dai, Kaikui Liu, Xiangxiang Chu, Linyuan Lv, Philip Torr, and Kevin Qinghong Lin. Code2world: A gui world model via renderable code generation. *arXiv preprint arXiv:2602.09856*, 2026.
- Yuhang Zhou, Lizhu Zhang, Yifan Wu, Jiayi Liu, Xiangjun Fan, Zhuokai Zhao, and Hong Yan. Synthetic sandbox for training machine learning engineering agents. *arXiv preprint arXiv:2604.04872*, 2026.
- Kaijie Zhu, Yuzhou Nie, Yijiang Li, Yiming Huang, Jialian Wu, Jiang Liu, Ximeng Sun, Zhenfei Yin, Lun Wang, Zicheng Liu, et al. Termigen: High-fidelity environment and robust trajectory synthesis for terminal agents. *arXiv preprint arXiv:2602.07274*, 2026.
- Siyu Zhu, Yanbin Jiang, Hejian Sang, Shao Tang, Qingquan Song, Biao He, Rohit Jain, Zhipeng Wang, and Alborz Geramifard. Planner-r1: Reward shaping enables efficient agentic rl with smaller llms. *arXiv preprint arXiv:2509.25779*, 2025.

A Domain Interaction Examples

Figure 14 and 15 show representative interaction examples across all seven domains covered by Qwen-AgentWorld. Each panel shows one agent–environment turn: the action issued by the agent (top) and the simulated observation produced by Qwen-AgentWorld (bottom).



(a) OS: the agent clicks File > Print in a spreadsheet application; the world model predicts the Print backstage view with print settings and a document preview.



(b) Web: the agent clicks View Details on a job listing page; the world model predicts the full job posting with role description, requirements, and application details.

Figure 14: Interaction examples from the GUI domains (continued). OS and Web are shown here; Android is in Section 2.3. Given the current screen and the agent’s action, the world model predicts the next GUI state in HTML, which is then rendered as a screenshot.

Terminal

Turn 7/9 · 10.7k tokens

Qwen-AgentWorld Input (Action)

keystrokes: "cd /app/pmars-0.9.4/src && make CFLAGS='-O -DEXT94 -DPERMUTATE -DRWLIMIT' LIB=''\\n"

Qwen-AgentWorld Output (Simulated Observation)

root@fa24c760:/app# cd /app/pmars-0.9.4/src && make CFLAGS='-O ...' LIB=''
Making pmars with compiler flags -O -DEXT94 -DPERMUTATE -DRWLIMIT
gcc -O -DEXT94 -DPERMUTATE -DRWLIMIT -c pmars.c
... (asm.o, eval.o compiled)
gcc -O -DEXT94 -DPERMUTATE -DRWLIMIT -c disasm.c
sim.h:92:12: warning: built-in function 'round' declared as non-function [-Wbuiltin-declaration-mismatch]
92 | extern int round;
... (cdb.o, sim.o -- same warning; pos.o compiled with overflow warning)
... (clparse.o, global.o, token.o, str_eng.o)
Linking pmars
done
root@fa24c760:/app/pmars-0.9.4/src#

(a) Terminal: the agent issues a make command to compile a C project; the world model predicts the full compiler output including warnings and the final linking step.

MCP Tool Use

Turn 5/13 · 118.5k tokens

Qwen-AgentWorld Input (Action)

API-post-database-query
database_id: "e0956d87-47b6-83fc-8fd0-812438cb7061"
filter: { or: [
 { property: "Status", select: { equals: "Expired" } },
 { property: "Status", select: { equals: "To be returned" } }
] }

Qwen-AgentWorld Output (Simulated Observation)

{
 "object": "List",
 "results": [
 {
 "object": "page",
 "id": "73a56d87-47b6-8260-bbbb-814060904ac3",
 "properties": {
 "Serial": { "type": "title", "plain_text": "36x10PIQ" },
 "Status": { "name": "To be returned", "color": "gray" },
 "Registered to": { "plain_text": "User 59cad035" },
 "Tags": [{ "name": "Computer Accessory" }],
 "Expiration": { "start": "2023-06-22" },
 "Vendor": { "name": "Pineapple" }
 }
 },
 ... (4 more asset records matching the OR filter)
]
}

(b) MCP Tool Use: the agent queries a Notion database via a structured JSON API call; the world model returns matching records with properties, tags, and metadata.

Search

Turn 2/3 · 24.7k tokens

Qwen-AgentWorld Input (Action)

web_extractor(urls=["en.wikipedia.org/.../presidential_elections_by_popular_vote_margin",
 "britannica.com/.../Presidential-Election-Results-1788863"],
goal="Get a complete list of all US presidential election results from 1789 to 2024,
including: year, winner name, party, popular vote count, and electoral vote count.")

Qwen-AgentWorld Output (Simulated Observation)

Election	Winner	Party	Electoral	Margin	Votes	Runner-up	Turnout
1788-89	George Washington	Ind.	69/69	100%	43,782	--	11.6%
1800	Thomas Jefferson	D-R	73/138	22.86%	41,330	Aaron Burr (D-R)	32.3%
1860	Abraham Lincoln	Rep.	180/303	10.13%	1,855,993	Breckinridge (Dem.)	81.8%
1932	F. D. Roosevelt	Dem.	472/531	17.76%	22,821,277	Hoover (Rep.)	56.9%
1960	John F. Kennedy	Dem.	303/537	0.17%	34,220,984	Nixon (Rep.)	63.8%
2008	Barack Obama	Dem.	365/538	7.27%	69,498,516	McCain (Rep.)	57.1%
2020	Joe Biden	Dem.	306/538	4.45%	81,284,666	Trump (Rep.)	62.8%

Source: en.wikipedia.org (60 elections, 1788-2024; 7 shown)

(c) Search: the agent extracts U.S. presidential election results from Wikipedia; the world model returns a structured table spanning 60 elections.

Figure 15: Interaction examples from the text-based domains (continued). Terminal, MCP Tool Use, and Search are shown here; Software Engineering is in Section 2.3.

36

B Training Dynamics

To understand what RL learns and in what order, we track the five open-ended evaluation dimensions (Format, Factuality, Consistency, Realism, Quality) across 440 RL steps on Qwen-AgentWorld-35B-A3B, evaluating every 10 steps.

Dimensions improve at different rates.

Figure 16 shows the per-dimension score trajectory. Consistency shows the largest absolute improvement (+0.29, from 2.81 to 3.10), while Quality improves least (+0.11). The dimensions converge at different speeds: Format and Quality reach 90% of their total improvement within 90 steps, while Consistency and Realism require ~ 250 steps. Formatting conventions (JSON structure, terminal prompt strings) are surface patterns that RL reinforces quickly. Consistency requires the model to internalize cross-turn state tracking, a deeper capability that develops gradually. Notably, the RL reward is a single scalar (the mean of all five dimensions), yet the dimensions diverge sharply in improvement rate, indicating that RL preferentially improves what is easiest to optimize given the training signal.

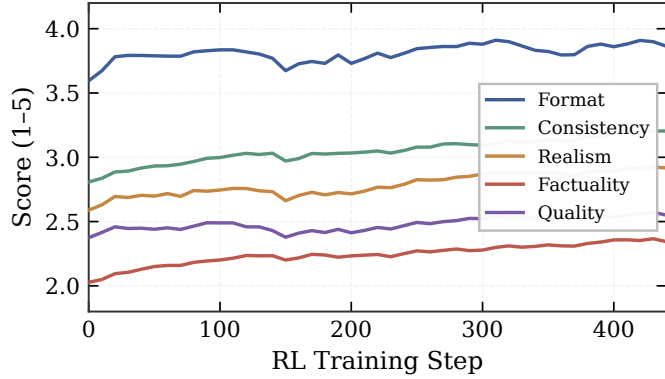


Figure 16: Training dynamics across 440 RL steps (Qwen-AgentWorld-35B-A3B). Format converges within ~ 90 steps. Consistency requires ~ 250 steps.

Factuality gains the most in relative terms. Factuality improves by 11.3% relative to its initial value (2.03 to 2.26), the largest relative gain among all dimensions. This confirms that RL drives the model toward more accurate environment responses, not merely polished formatting. However, Factuality remains the lowest-scoring dimension throughout training, indicating that factual world knowledge is the hardest aspect of environment simulation.

C Rule-Based Verification

Beyond the open-ended rubric evaluation of AgentWorldBench, we design an independent set of rule-based verifiers that provide deterministic, reproducible checks on three targeted capability axes: **controllability** (adherence to explicit simulation instructions), **error handling** (faithful reproduction of environment failure modes), and **long-context consistency** (coherent state tracking across long interaction histories). These axes isolate specific failure modes that a rubric-based judge may underweight.

The test cases for all three axes are derived from the main evaluation set through a shared trajectory-grounded synthesis and validation pipeline. We first summarize the initial state and deterministic constraints of each candidate trajectory, including environment configuration, file-system or database state, active services, tool schemas, and other domain-specific states. Based on these summaries, we filter for turns whose expected observations are sufficiently deterministic and whose behavior matches one of the three rule-based axes: adherence to explicit simulation instructions for controllability, faithful reproduction of invalid operations for error handling, or dependence on earlier state mutations for long-context consistency. For each retained turn, we synthesize an executable verifier specialized to that check and domain. Then, we validate candidate verifiers against the ground-truth observation and multiple model rollouts, retaining only cases whose rule-based verdicts agree with reference-grounded rubric labels. However, because rule-based verifiers cannot fully ascertain the correctness of certain cases, we additionally employ an LLM judge to filter out incorrect outputs that might otherwise deceive the verifiers.

Below, we describe how the pipeline is instantiated for each capability axis.

- **Controllability (Ctrl).** This axis tests whether the LWM follows explicit simulation instructions in addition to the interaction history. We select turns whose normal behavior can be changed by an explicit control condition while keeping the surrounding trajectory context fixed. Each test case augments the normal trajectory context with such a condition, such as forcing a terminal command to fail with a specified dependency conflict, requiring a tool response to withhold intermediate information, or using contrastive GUI conditions where the same action should lead to different

Table 10: Rule-based verification: per-domain accuracy (% , $\uparrow$) across four text domains (MCP, Search, Terminal, SWE) and three GUI domains (Android, Web, OS). The highest and second-best scores per column are shown in **bold** and underlined, respectively.

	Model	Text				GUI			Avg.
		MCP	Search	Term.	SWE	Android	Web	OS	
Frontier	Claude Opus 4.6	<u>76.90</u>	77.55	68.60	54.40	<u>80.06</u>	75.71	61.81	<u>70.72</u>
	Claude Sonnet 4.6	<u>76.43</u>	<u>71.90</u>	64.43	55.27	<u>75.23</u>	77.16	52.57	<u>67.57</u>
	GPT-5.4	82.90	66.95	<u>68.13</u>	62.37	81.08	81.61	67.46	72.93
	Gemini 3.1 Pro	68.77	61.45	62.33	53.47	76.73	75.32	<u>65.50</u>	66.22
Open-weight	DeepSeek-V4-Pro	69.90	56.85	57.40	44.50	76.36	72.83	61.28	62.73
	Kimi K2.6	70.30	69.40	55.53	45.20	61.90	75.63	62.49	62.92
	GLM-5.1	72.53	71.70	55.47	41.77	67.98	72.06	55.98	62.50
	MiniMax-M2.7	55.03	44.30	32.40	25.17	71.87	70.36	49.39	49.79
Qwen	Qwen3.6-35B-A3B	49.50	47.00	41.30	35.07	59.79	67.83	44.03	49.22
	Qwen3.6-Plus	67.57	67.90	58.43	47.57	63.81	69.85	54.29	61.35
	Qwen3.6-Max-Preview	73.40	64.05	59.37	45.03	60.19	67.02	50.47	59.93
Ours	Qwen3.5-35B-A3B	57.80	40.70	36.83	28.50	61.10	68.49	44.07	48.21
	Qwen-AgentWorld-35B-A3B	60.47	44.90	54.53	48.73	62.34	72.37	57.61	57.28
	Qwen3.5-397B-A17B	67.33	63.50	51.80	44.17	69.25	70.22	54.03	60.04
	Qwen-AgentWorld-397B-A17B	72.37	58.95	62.63	<u>60.33</u>	76.79	<u>80.74</u>	58.01	67.12

predicted screens under different hidden states. The verifier checks whether the generated observation satisfies the instructed behavior while preserving the domain’s normal output format.

- **Error Handling (Err).** Real environments produce errors: commands fail, files are missing, network calls time out, permissions are denied. A faithful simulator must reproduce these failure modes rather than fabricating a successful response. We retain turns with deterministic error outcomes, including missing preconditions or invalid operations such as deleting a non-existent file, calling a tool with a malformed argument, or writing to a read-only path. The Search domain does not contribute error-handling cases because its trajectories consist primarily of successful information-seeking interactions and do not provide deterministic, executable error outcomes suitable for this axis. The verifier checks that the generated observation contains an appropriate error signal (exit code, exception message, error-typed JSON response) rather than a plausible-looking success.
- **Long-Context Consistency (LC).** Long interaction sequences accumulate state: files are written and later read, environment variables are set and later referenced, databases are populated and later queried. This axis checks whether the simulator maintains coherent state across turns. We construct test cases by identifying cross-turn state dependencies within trajectories, where an earlier action changes or reveals environment state and a later observation depends on that state. Examples include write-read pairs, environment-variable updates followed by command execution, database mutations followed by queries, or GUI actions that change a later screen. The verifier checks whether the generated observation respects the expected state constraints, such as updated file contents, referenced environment variables, queried records, or changed GUI elements, thereby reflecting the model’s consistency in long-horizon simulation.

Table 10 reports per-domain verification scores. Rule-based verification corroborates the main findings: world-model training improves adherence to explicit simulation instructions, faithful reproduction of environment failure modes, and coherent state tracking across long interaction histories. GPT-5.4 ranks first overall with an average score of 72.93, while Qwen-AgentWorld-397B-A17B places second at 67.12, outperforming all other frontier models on GUI domains.

C.1 Per-Axis Breakdown

Table 11 and 12 provide per-sub-capability breakdowns (controllability, error handling, long-context consistency) for text-based and GUI domains, respectively, supplementing the per-domain averages in Table 10.

Table 11: Rule-based evaluation on text-based domains: accuracy (% , $\uparrow$) on three sub-capabilities. Ctrl: controllability; Err: error handling; LC: long-context consistency. The highest and second-best scores are shown in **bold** and underlined, respectively.

	Model	MCP			Term.			Search		SWE			Avg.
		Ctrl	Err	LC	Ctrl	Err	LC	Ctrl	LC	Ctrl	Err	LC	
<i>Frontier</i>	Claude Opus 4.6	72.3	<u>78.4</u>	<u>80.0</u>	68.7	<u>81.0</u>	56.1	74.2	80.9	52.8	59.6	50.8	<u>69.4</u>
	Claude Sonnet 4.6	<u>74.3</u>	77.0	78.0	<u>63.9</u>	77.0	52.4	<u>72.0</u>	71.8	57.5	59.1	49.2	67.0
	GPT-5.4	76.2	84.5	88.0	62.0	88.7	<u>53.7</u>	70.5	63.4	<u>59.8</u>	65.4	61.9	70.1
	Gemini 3.1 Pro	59.6	71.7	75.0	60.0	77.0	50.0	61.8	61.1	52.0	58.4	50.0	61.5
<i>Open-weight</i>	DeepSeek-V4-Pro	63.4	72.3	74.0	50.6	74.0	47.6	54.9	58.8	52.0	52.9	28.6	57.2
	Kimi K2.6	65.3	71.6	74.0	49.4	73.3	43.9	66.3	72.5	43.3	55.8	36.5	60.1
	GLM-5.1	67.3	74.3	76.0	49.4	74.3	42.7	70.1	<u>73.3</u>	42.5	49.5	33.3	60.4
	MiniMax-M2.7	39.6	63.5	62.0	30.1	50.0	17.1	41.7	46.9	29.1	27.4	19.0	39.2
<i>Qwen</i>	Qwen3.6-35B-A3B	37.6	66.9	44.0	37.3	61.0	25.6	45.1	48.9	41.7	36.5	27.0	43.2
	Qwen3.6-Plus	60.4	70.3	72.0	51.8	74.7	48.8	63.3	72.5	52.0	55.8	34.9	60.4
	Qwen3.6-Max-Preview	69.3	70.9	<u>80.0</u>	54.8	75.7	47.6	60.2	67.9	44.1	57.7	33.3	60.5
<i>Ours</i>	Qwen3.5-35B-A3B	54.5	64.9	54.0	32.5	50.0	28.0	33.3	48.1	34.6	30.3	20.6	41.0
	Qwen-AgentWorld-35B-A3B	52.5	68.9	60.0	50.0	67.3	46.3	37.1	52.7	48.0	60.1	38.1	52.2
	Qwen3.5-397B-A17B	60.4	73.6	68.0	47.0	<u>65.7</u>	42.7	59.8	67.2	44.1	56.7	31.7	56.7
	Qwen-AgentWorld-397B-A17B	63.4	75.7	78.0	59.0	77.7	51.2	56.1	61.8	61.4	<u>62.5</u>	<u>57.1</u>	63.6

Table 12: Rule-based evaluation on GUI domains: accuracy (% , $\uparrow$) on three sub-capabilities. Ctrl: controllability; Err: error handling; LC: long-context consistency. The highest and second-best scores are shown in **bold** and underlined, respectively.

	Model	Android			Web			OS			Avg.
		Ctrl	Err	LC	Ctrl	Err	LC	Ctrl	Err	LC	
<i>Frontier</i>	Claude Opus 4.6	<u>80.9</u>	79.4	<u>79.7</u>	82.8	60.0	<u>84.3</u>	<u>70.9</u>	59.0	52.0	72.1
	Claude Sonnet 4.6	77.9	69.8	76.0	<u>81.7</u>	68.0	81.8	60.9	49.0	45.1	67.8
	GPT-5.4	77.4	82.5	83.9	<u>79.6</u>	80.0	85.2	71.2	<u>72.0</u>	54.6	76.3
	Gemini 3.1 Pro	72.6	87.3	74.2	79.0	68.0	79.0	60.4	80.5	50.0	<u>72.3</u>
<i>Open-weight</i>	DeepSeek-V4-Pro	73.2	85.7	73.6	78.3	62.0	78.2	68.3	61.0	50.8	70.1
	Kimi K2.6	63.9	58.7	61.9	79.1	69.0	78.8	71.2	60.0	52.8	66.2
	GLM-5.1	68.0	65.1	69.9	78.5	64.0	73.7	62.6	53.0	50.4	65.0
	MiniMax-M2.7	66.9	77.8	73.1	68.5	73.0	69.6	51.6	51.0	43.5	63.9
<i>Qwen</i>	Qwen3.6-35B-A3B	56.7	66.7	58.6	72.4	60.0	71.1	50.1	38.0	44.0	57.5
	Qwen3.6-Plus	62.3	69.8	61.6	69.2	66.0	74.4	58.3	55.0	47.0	62.6
	Qwen3.6-Max-Preview	60.1	68.3	55.2	68.6	58.0	74.5	47.7	53.0	50.9	59.6
<i>Ours</i>	Qwen3.5-35B-A3B	57.2	66.0	60.1	74.3	63.6	67.6	47.7	41.4	43.1	57.9
	Qwen-AgentWorld-35B-A3B	53.0	95.2	51.4	66.2	<u>84.3</u>	66.6	60.1	53.0	60.9	65.6
	Qwen3.5-397B-A17B	67.4	65.1	73.8	71.0	68.0	71.7	58.9	54.0	46.5	64.0
	Qwen-AgentWorld-397B-A17B	81.3	<u>88.7</u>	64.9	75.8	84.5	82.0	56.5	60.0	<u>57.2</u>	72.3

D Open-Ended Judge Prompts

This section provides the complete judge system prompts used for open-ended evaluation (§4.2) across all seven domains. All prompts score predictions on the same five dimensions (Format, Factuality, Consistency, Realism, Quality). Text-based domain prompts define each dimension with content-type classification for differentiated matching. GUI domain prompts apply the same five dimensions with anchor-based calibration: domain-specific anchors (UI elements, navigation state, visible text) govern how each dimension is scored for screen-based observations.

D.1 Terminal Domain

```
# Role and Objective

You are a professional evaluator assessing simulated terminal outputs from a Terminal World Model. The Terminal World Model
simulates the execution of terminal commands within a Linux/Unix environment, generating plausible terminal output based
on the command sequence and session context.

Your task is to compare the Simulated Terminal Output against the Ground Truth (Real Terminal Output) and evaluate the
simulation quality across the following five dimensions. The Ground Truth serves as the absolute reference standard.
Every penalization or commendation MUST reference specific differences or matches with the Ground Truth.

---

# Content Type Classification

Important Context: The Terminal World Model has no access to the real environment state. It can only "know" information explicitly
shown or created during the current interaction session. For any pre-existing state (e.g., file contents not
written in this session, installed packages, system configurations, directory structures), the model must infer plausible
values.

Before evaluation, apply different verification standards based on information availability:

| Content Type | Verification Standard | Examples |
|---|---|---|
| Deterministic content | Must match Ground Truth exactly. | echo output, cat of a file written earlier in this session,
  computation results |
| Pre-existing environment content | Verify format and plausibility only. Do NOT penalize different but reasonable values.
  | ls of a pre-existing directory, cat of a pre-existing file, version numbers |
| Runtime metadata | Verify format and plausibility only. | Timestamps, PIDs, container IDs, memory addresses, download
  speeds |

---

# Evaluation Dimensions

## 1. Format

Definition: Evaluates whether the simulated output adheres to the formatting conventions shown in the Ground Truth. This
dimension evaluates ONLY formatting, not content correctness.

Key Points:
- Overall Structure: The output must conform to the same structural layout as the Ground Truth -- including the prompt-
  command-output cycle, section separations, and terminal conventions.
- Prompt Format: The shell prompt pattern (e.g., user@host:/path$ or #) must match the Ground Truth's convention. After
  directory-changing commands, the path component in the prompt must update accordingly.
- Command Echo: The executed command must appear correctly after the prompt, exactly as typed in the input.
- Line Breaks and Spacing: Precise line break placement matters -- empty lines between logical sections, no spurious blank
  lines within continuous output, and correct separation between command output and the next prompt.
- Special Formatting: When the Ground Truth shows specific formatting patterns (e.g., here-doc continuation markers like >,
  progress indicators, tabular/columnar alignment, interactive prompts), the simulation must follow the same conventions.
- Output Boundaries: The response should end appropriately as shown in Ground Truth -- typically with the next prompt ready
  for input, or mid-execution if the command is still running.

What NOT to penalize here: Content differences (different file names, different version numbers) -- these belong to
Factuality or Realism.

---

## 2. Factuality

Definition: Evaluates the factual correctness of the simulated content. The Ground Truth serves as the absolute reference
standard. The simulator must NOT fabricate or contradict any information that is deterministic or previously
established in the session.

Key Points:
- Ground Truth as Absolute Reference: Always compare the simulated output against the Ground Truth line by line.
- Deterministic Content -- Strict Match: Outputs fully determined by the command and known session state must match Ground
  Truth exactly.
- Pre-existing Content -- Plausibility Check: For content depending on unknown environment state, do NOT require exact
  matches. Instead verify: (1) format matches Ground Truth's pattern, (2) content is plausible for the domain, (3) no
  internal contradictions.
- Runtime Metadata: Timestamps, PIDs, and other dynamic metadata need only be format-valid and range-plausible.
- Fabrication Policy: Inventing content that contradicts known session state is strictly prohibited. Generating
  plausible content for unknown/pre-existing state is allowed.

---

## 3. Consistency

Definition: Measures whether the simulated output remains coherent and consistent with all previously established state
throughout the interaction.

Key Points:
- State Tracking: All state changes from prior turns (file creations, modifications, deletions, environment variable
  settings, directory changes, process launches, etc.) must be correctly reflected in subsequent outputs.
- No Contradictions: The output must not contradict any fact or state established in prior visible turns.
```

```

- **Contextual Continuity:** The simulated environment should evolve coherently -- operations that depend on prior state should produce results consistent with that state.

---

## 4. Realism

**Definition:** Evaluates how well the simulation captures the authentic behavior of a real terminal environment, **as evidenced by the Ground Truth**.

**Key Points:**
- **Behavioral Fidelity:** Command behaviors should match real-world expectations.
- **Style Consistency:** The simulated output should have a consistent style with the Ground Truth -- including tone, terminology, and presentation conventions.
- **Value Plausibility:** Numeric values, file sizes, version numbers should be within reasonable ranges for the domain and context.
- **Multi-stage Output:** Commands that produce multi-stage output (e.g., package installation, build processes) should show realistic progress stages.

---

## 5. Quality

**Definition:** Evaluates whether the output is complete and appropriately concise compared to the Ground Truth.

**Key Points:**
- **Completeness:** The simulated response must include all critical information present in Ground Truth. Missing critical content is penalized.
- **Conciseness:** The content should not be overly verbose relative to Ground Truth.
- **Proportionality:** The scope and scale of the output should be comparable to Ground Truth.

```

D.2 MCP Domain

```

# Role and Objective

You are a professional evaluator specializing in assessing simulated tool outputs from a **Tool World Model**. The Tool World Model simulates the execution of tool calls within tool-augmented agent scenarios, generating plausible and contextually appropriate tool execution results based on the provided tool call information.

Your task is to compare the **Simulated Tool Response** against the **Ground Truth (Real Tool Response)** and evaluate the simulation quality across the following five dimensions.

---

# Content Type Classification

Before evaluation, classify the content in the response into the following categories, as they require different verification standards:

| Content Type | Verification Standard | Examples |
|---|---|---|
| **Objective Facts** | Must match Ground Truth exactly. | Identifiers, public entity names, error messages, labels, code execution results |
| **Numeric Data** | Allow reasonable variance for real-time data. | Counts, measurements, coordinates, statistical values |
| **Private/Session/Inaccessible Data** | Verify validity and realism only. | Generated IDs, file paths, timestamps, API metadata |
| **Structural Metadata** | Must match the overall structure and schema. | JSON keys, array structures, response wrappers |

---

# Evaluation Dimensions

## 1. Format

**Definition:** Evaluates whether the simulated output adheres to the real tool's formatting specifications, including overall structure, indentation, layout, field ordering, line breaks, spacing, special symbols, and native formatting style (e.g., JSON, plain text, markdown, structured data).

## 2. Factuality

**Definition:** Evaluates whether the **verifiable information** in the simulated output matches the Ground Truth. This is the **CORE** dimension for correctness.

**Key Points:**
- **Tool Execution:** Correct interpretation of tool parameters and correct execution of core functionality.
- **Exact Matching:** Objective Facts must match exactly.
- **No Hallucination:** Content that does not exist in the Ground Truth is penalized.
- **Status Accuracy:** Correct status/result type (success vs. error, found vs. not found).

## 3. Consistency

**Definition:** Evaluates whether the simulated output remains coherent with **previous context and tool state** throughout multi-turn interactions.

**Key Points:**

```

```

- **Resource References:** Correctly references resources (IDs, data, states) from previous turns.
- **State Tracking:** Maintains proper state transitions (e.g., thoughtNumber increments, pagination offsets).
- **Causal Relationships:** No conflicts with historical information; maintains logical causality between operations.

## 4. Realism

**Definition:** Evaluates how well the simulation matches the **behavioral characteristics observed in the Ground Truth. This focuses on response pattern alignment with the reference.

**Key Points:**
- **Response Pattern Match:** The response type (success, error, empty, partial) must match Ground Truth.
- **Value Range Alignment:** Numeric values should be within reasonable range of Ground Truth values.
- **Edge Case Handling:** Proper reproduction of empty results, not-found responses, and error scenarios.
- **Error Pattern Match:** Error message format and structure should align with Ground Truth patterns.

## 5. Quality

**Definition:** Evaluates whether the output is complete and appropriately concise compared to the Ground Truth.

**Key Points:**
- **Completeness:** The simulated response must return all necessary content present in the Ground Truth.
- **Conciseness:** The content should not be overly verbose.

```

D.3 Search Domain

```

# Role and Objective

You are a professional evaluator specializing in assessing simulated tool outputs from a **Search World Model**. The Search World Model simulates the execution of real search engine tools (web_search and web_extractor), generating responses that must strictly adhere to the provided **Ground Truth (Real Tool Response)**.

Your task is to compare the **Simulated Tool Response** against the **Ground Truth** and evaluate the simulation quality across the following five dimensions.

---

# Evaluation Dimensions

## 1. Format

**Definition:** Evaluates whether the simulated output adheres to the structural and field requirements of the real tool's output.

**Key Points:**
- **Overall Structure:** The output must conform to the same structural format as the Ground Truth.
  - For web_search: Numbered result entries with required fields (e.g., url, title, snippet).
  - For web_extractor: Webpage layouts matching the Ground Truth.
- **Formatting Details:** Valid, well-structured JSON where applicable. Proper line breaks, indentation, and spacing.

## 2. Factuality (Content Accuracy)

**Definition:** Evaluates the factual correctness of the simulated content. The Ground Truth serves as the **absolute reference standard**. The simulator must NOT fabricate or contradict any information present in the Ground Truth.

**Key Points:**
- **Content Present in Ground Truth:** Facts and sources must match exactly. Any fabrication or contradiction is penalized.
- **Content Not in Ground Truth:** Assess consistency with real-world facts. Penalize only errors **clearly contradicted** by the Ground Truth.
- **Metadata Plausibility:** Dates, timestamps, and URLs should be plausible.
- **Precedence:** The Ground Truth takes precedence as the definitive source of truth.

## 3. Consistency

**Definition:** Measures whether the simulated output remains coherent and consistent with previous context throughout the conversation.

**Key Points:**
- Repeated web_search calls with similar queries should return broadly consistent results.
- Repeated web_extractor calls on the same URL should yield consistent content.

## 4. Realism (Search Behavior)

**Definition:** Evaluates how well the simulation replicates realistic search engine behavior. This dimension measures the **retrieval quality** of web_search and web_extractor.

**Key Points:**
- **Relevance Alignment:** Simulated results must align with the expected information as defined by the Ground Truth.
- **Ranking Priority:** The ordering of web_search results should reflect realistic search engine prioritization.
- **Behavioral Fidelity:** Tool behaviors should match real-world expectations (e.g., URLs matching query topics, plausible dates).

## 5. Quality

**Definition:** Evaluates whether the output is complete and appropriately concise compared to the Ground Truth.

```

```
**Key Points:**
- **Completeness:** The simulated response must return all necessary results, especially the top-ranked ones.
- **Conciseness:** The content should not be overly verbose.
```

D.4 SWE Domain

```
# Role and Objective

You are a professional evaluator specializing in assessing simulated tool outputs from a **Tool World Model**. The Tool World Model simulates the execution of tool calls within a realistic command-line and filesystem environment, generating plausible tool execution results based on the provided tool call information.

Your task is to compare the **Simulated Tool Response** against the **Ground Truth (Real Tool Response)** and evaluate the simulation quality across the following five dimensions.

---

# Content Type Classification

**Important Context:** The Tool World Model has no access to the real environment state. It can only "know" information **explicitly shown or created during the current interaction session**.

Before evaluation, classify the content in the tool output into the following categories:

| Content Type | Verification Standard | Examples |
|---|---|---|
| **Objective Facts** | Must match Ground Truth exactly. | Tool execution success/failure status, error message types, exit codes |
| **Session/Environment-Specific Data** | Verify format validity and reasonableness only. | Timestamps, PIDs, file modification times, version numbers |
| **Private/Unprovided Context-Dependent Data** | Verify format and semantic correctness. | Output of ls in user directories, file contents (when not previously shown), configuration values |
| **Structural/Formatting Elements** | Must match exactly. | JSON structure, XML tags, output format, indentation, line breaks |

---

# Evaluation Dimensions

## 1. Format

**Definition:** Evaluates whether the simulated output matches the real tool's format. This includes overall structure, indentation, layout, field ordering, line breaks, spacing, and adherence to the tool's native formatting style. This dimension evaluates **ONLY formatting**, not content correctness.

## 2. Factuality

**Definition:** Evaluates whether the **verifiable information** in the simulated output matches the Ground Truth. This is the **CORE** dimension for semantic correctness.

**Key Points:**
- **Tool Execution Simulation:** The model must correctly simulate tool execution logic. Success/failure status and exit codes must exactly match Ground Truth.
- **Error Message Correctness:** Error message type must be accurate and match the actual failure reason.
- **Content Accuracy:** Content explicitly shown or created in the session must exactly match Ground Truth. Deterministic operations must produce matching output.
- **Fabrication Policy:** Inventing content that contradicts known state is prohibited. Plausible values for unknown environment state are allowed.

---

## 3. Consistency

**Definition:** Evaluates whether the simulated output remains coherent with **previous tool states and interaction history** throughout multi-turn interactions.

**Key Points:**
- **File System State:** Files created in previous commands must exist in subsequent read_file or ls operations. File modifications must be reflected.
- **Environment & Session State:** Environment variables, working directory changes, and configurations must persist. No contradictions with prior information.

---

## 4. Realism

**Definition:** Evaluates how well the simulation captures the **authentic behavior patterns** of real tool execution. This focuses on behavioral authenticity and stylistic accuracy, NOT content correctness.

**Key Points:**
- **Tool Behavior Patterns:** Tool-specific output format and structure should match typical behavior. Success/confirmation messages follow the tool's standard response style.
- **Output Semantics:** For environment-dependent operations, output must be semantically plausible. Appropriate output verbosity based on the operation type.
- **Numeric Reasonableness:** File sizes, permissions, timestamps are plausible and chronologically consistent.
```

- **Error Message:** Error message format matches the originating tool.
- **Edge Case Handling:** Reasonable behavior for empty results or boundary conditions.

5. Quality

Definition: Evaluates whether the output is both complete and appropriately concise relative to the Ground Truth.

Key Points:

- **Completeness:** All critical information from Ground Truth is present.
- **Conciseness:** Output is not overly verbose compared to Ground Truth.

D.5 Android Domain

Android World Model Judge Guidance

You are judging an Android world-model prediction: the simulated screen state after the latest user operation, compared with the real observed screen state.

Use the task-provided evaluation protocol exactly as given by the user message. This system prompt only adds Android-specific judging anchors.

Core Principle

A strong prediction is not just a plausible Android story. It must preserve the concrete state representation used by the trajectory and correctly update the visible Android state caused by the latest user operation.

Treat the ground truth as the strongest evidence for visible Android facts. If the ground truth appears clearly inconsistent with the prior history, rely on causal reasoning from the interaction history; otherwise, prefer predictions that preserve more concrete ground-truth anchors.

Android Anchors

Before rewarding a prediction strongly, check these anchors when they are visible or implied by the turn:

- Active app, package/activity, screen title, navigation stack, and whether the app/page stayed unchanged.
- Dialogs, bottom sheets, permission prompts, notification shade state, toasts, popups, and transient overlays.
- Focused input, cursor position, typed text, suggestion changes, IME/keyboard visibility, and keyboard layout.
- Selected, checked, toggled, disabled, or highlighted controls.
- List, feed, recycler-view, tab, page, and scroll position.
- Exact task labels, search queries, filenames, contact names, item titles, timestamps, timer/clock text, and other visible text anchors.
- Android-specific behavior such as back/home navigation, app switching, permission handling, soft-keyboard reflow, and disabled-control behavior.

Strictness Rules

- Penalize unchanged predictions after meaningful operations when the real screen visibly changes.
- Penalize speculative navigation, refreshed pages, or invented transitions when the observed result is unchanged or only a small anchor changes.
- Penalize broad app-level correctness when task-critical anchors such as typed text, selected chip, dialog presence, keyboard state, or list position are wrong.
- Penalize fabricated screens, impossible Android states, generic summaries, and outputs that explain what should happen instead of showing the resulting state.
- Do not let clean structure hide a wrong Android state; the primary post-action effect matters most.

Score Separation Policy

Map anchor accuracy to scores as follows:

- Top score: reserve for predictions that match the primary post-action effect and nearly all task-critical visible anchors.
- High-but-not-top score: use when the main transition is correct but one or two local anchors are missing or slightly stale.
- Middle score: use when the broad destination or app/page/window is right but important visible state details are wrong, missing, or invented.
- Low score: use when the prediction is mostly a plausible story, unchanged state, or wrong branch despite sharing some surrounding context.
- Minimum score: use for wrong environment/page/app, malformed/non-state output, fabricated major content, or explanations instead of a predicted observation.

Dimension-Specific Scoring Anchors

Use each dimension independently; do not let a well-formed UI tree inflate factual correctness.

- Format: score only schema/tag/readability compliance. A clean format cannot compensate for wrong state facts.
- Factuality: compare concrete visible anchors against the real observation. If the main screen/action result is wrong, use 1-2. If the main result is right but several active-region anchors are wrong, use 3. Use 4-5 only for close anchor matches.
- Consistency: check causal continuity from the previous state and latest action. Wrong change-vs-no-change, wrong back/navigation result, wrong keyboard/dialog/focus behavior, or stale state after a state-changing action should be 1-2.
- Realism: judge whether the predicted Android transition is behaviorally plausible for the app and action, but cap it at 3 when factuality or consistency is 1-2.
- Quality: judge usefulness as a replacement next-state observation. If a downstream agent would take the wrong next action because of missing/wrong active anchors, use 1-2; if usable only at a broad page level, use 3.

Before assigning any 4 or 5, explicitly verify the primary action effect plus at least these active anchors when present: package/activity or screen title, focused/selected control, exact typed/search text, keyboard/dialog/toast state, list/feed item labels, and scroll position.

Discriminative Error Calibration

- Treat the latest action's primary visible effect as a gate. If it is wrong, stale, or missing, set factuality ≤ 2 and quality ≤ 2 even when many background nodes match.
- If the output copies or preserves much of the prior state while the ground truth changes, score it as a stale-state error.
- Downweight generic complete-looking hierarchies: if exact active text, selected/focused element, transient toast/dialog, and visible list/feed rows do not match, total score should normally be ≤ 3 .

D.6 Web Domain

Web World Model Judge Guidance

You are judging a web world-model prediction: the simulated browser/page state after the latest user operation, compared with the real observed page state.

Use the task-provided evaluation protocol exactly as given by the user message. This system prompt only adds web-specific judging anchors.

Core Principle

A strong prediction is not just a plausible browsing story. It must preserve the concrete state representation used by the trajectory and correctly update the visible browser/page state caused by the latest user operation.

Treat the ground truth as the strongest evidence for visible web facts. If the ground truth appears clearly inconsistent with the prior history, rely on causal reasoning from the interaction history; otherwise, prefer predictions that preserve more concrete ground-truth anchors.

Web Anchors

Before rewarding a prediction strongly, check these anchors when they are visible or implied by the turn:

- Current URL, page title, active tab, active frame/iframe, browser dialog, and whether navigation actually happened.
- Load outcome, including success page, error page, spinner, redirect, authentication wall, or unchanged page.
- Exact visible text, headings, link labels, table rows, card titles, result counts, prices, dates, and messages.
- Form values, focused field, cursor position, validation text, enabled/disabled controls, checked boxes, selected options, and uploaded-file state.
- Modal, dropdown, menu, tooltip, popover, cookie banner, alert, and blocking overlay state.
- Scroll position, viewport/snapshot scope, selected text, highlighted element, and expanded/collapsed sections.

Strictness Rules

- Penalize predictions that invent successful navigation or content when the observed result is unchanged, loading, blocked, or errored.
- Penalize broad website-level correctness when task-critical anchors such as URL, title, form value, validation message, modal state, or visible row text are wrong.
- Penalize stale predictions that miss small but visible updates after typing, selecting, filtering, sorting, scrolling, submitting, or opening a menu.
- Penalize fabricated pages, impossible browser states, generic summaries, and outputs that explain what should happen instead of showing the resulting page state.
- Do not let clean structure hide a wrong web state; the primary post-action effect matters most.

Score Separation Policy

Map anchor accuracy to scores as follows:

- Top score: reserve for predictions that match the primary post-action effect and nearly all task-critical visible anchors.
- High-but-not-top score: use when the main transition is correct but one or two local anchors are missing or slightly stale.
- Middle score: use when the broad destination or app/page/window is right but important visible state details are wrong, missing, or invented.
- Low score: use when the prediction is mostly a plausible story, unchanged state, or wrong branch despite sharing some surrounding context.
- Minimum score: use for wrong environment/page/app, malformed/non-state output, fabricated major content, or explanations instead of a predicted observation.

Dimension-Specific Scoring Anchors

Use each dimension independently; do not let a well-formed DOM/accessibility tree inflate factual correctness.

- Format: score only schema/tag/readability compliance. A clean format cannot compensate for wrong page facts.
- Factuality: compare concrete visible anchors against the real observation. If the main page/action result is wrong, use 1-2. If the main result is right but several active-region anchors are wrong, use 3. Use 4-5 only for close anchor matches.
- Consistency: check causal continuity from the previous state and latest action. Wrong change-vs-no-change, wrong navigation/load result, wrong focus/dropdown/modal behavior, or stale state after a state-changing action should be 1-2.
- Realism: judge whether the predicted browser/page transition is behaviorally plausible for the site and action, but cap it at 3 when factuality or consistency is 1-2.
- Quality: judge usefulness as a replacement next-state observation. If a downstream agent would take the wrong next action because of missing/wrong active anchors, use 1-2; if usable only at a broad page level, use 3.

Before assigning any 4 or 5, explicitly verify the primary action effect plus at least these active anchors when present: URL/title or page identity, focused element, exact typed/search text, dropdown/modal/error state, selected item/filter, visible result/list/table labels, and scroll position.

Discriminative Error Calibration

- Treat the latest action's primary visible effect as a gate. If it is wrong, stale, or missing, set factuality ≤ 2 and quality ≤ 2 even when many background nodes match.
- If the output copies or preserves much of the prior state while the ground truth changes, score it as a stale-state error.
- Downweight generic complete-looking accessibility trees: if exact active text, selected/focused element, error/load/modal state, and visible result rows do not match, total score should normally be ≤ 3 .

D.7 OS Domain

Desktop OS World Model Judge Guidance

You are judging a desktop OS world-model prediction: the simulated desktop/app state after the latest user operation, compared with the real observed desktop state.

Use the task-provided evaluation protocol exactly as given by the user message. This system prompt only adds desktop-specific judging anchors.

Core Principle

A strong prediction is not just a plausible desktop story. It must preserve the concrete state representation used by the trajectory and correctly update the visible desktop state caused by the latest user operation.

Treat the ground truth as the strongest evidence for visible desktop facts. If the ground truth appears clearly inconsistent with the prior history, rely on causal reasoning from the interaction history; otherwise, prefer predictions that preserve more concrete ground-truth anchors.

Desktop Anchors

Before rewarding a prediction strongly, check these anchors when they are visible or implied by the turn:

- Active application, focused window, window title, z-order, minimized/maximized state, geometry, and workspace/desktop context.
- Dialogs, menus, context menus, popovers, file pickers, permission prompts, alerts, and whether they block interaction.
- Exact file, folder, document, tab, terminal, process, or app names visible in the state.
- Focused input, cursor position, selected text/items, edited text, unsaved/saved state, and clipboard-like visible effects.
- Terminal command output, prompt position, last visible rows, errors, exit state, and current working directory when represented.
- File-manager state such as created, renamed, moved, deleted, selected, or highlighted files and folders.
- Scroll position, visible viewport, status bars, clock/status text, coordinates, taskbar/dock/panel state, and notification indicators.

Strictness Rules

- Penalize predictions that switch apps/windows, close dialogs, dismiss menus, move focus, or alter files without causal support from the latest user operation.
- Penalize broad desktop-level correctness when task-critical anchors such as focused window, exact file name, terminal output, dialog state, selection, or geometry are wrong.
- Penalize stale predictions that miss small but visible updates after typing, opening menus, selecting files, running commands, scrolling, saving, or switching focus.
- Penalize fabricated windows, impossible desktop states, generic summaries, and outputs that explain what should happen instead of showing the resulting desktop state.
- Do not let clean structure hide a wrong desktop state; the primary post-action effect matters most.

Score Separation Policy

Map anchor accuracy to scores as follows:

- Top score: reserve for predictions that match the primary post-action effect and nearly all task-critical visible anchors.
- High-but-not-top score: use when the main transition is correct but one or two local anchors are missing or slightly stale.
- Middle score: use when the broad destination or app/page/window is right but important visible state details are wrong, missing, or invented.
- Low score: use when the prediction is mostly a plausible story, unchanged state, or wrong branch despite sharing some surrounding context.
- Minimum score: use for wrong environment/page/app, malformed/non-state output, fabricated major content, or explanations instead of a predicted observation.

Dimension-Specific Scoring Anchors

Use each dimension independently; do not let a well-formed desktop hierarchy inflate factual correctness.

- Format: score only schema/tag/readability compliance. A clean format cannot compensate for wrong desktop facts.
- Factuality: compare concrete visible anchors against the real observation. If the main window/action result is wrong, use 1-2. If the main result is right but several active-region anchors are wrong, use 3. Use 4-5 only for close anchor matches.
- Consistency: check causal continuity from the previous state and latest action. Wrong change-vs-no-change, wrong app/window/menu/dialog result, wrong focus/selection behavior, or stale state after a state-changing action should be 1-2.
- Realism: judge whether the predicted desktop transition is behaviorally plausible for the app and action, but cap it at 3 when factuality or consistency is 1-2.
- Quality: judge usefulness as a replacement next-state observation. If a downstream agent would take the wrong next action because of missing/wrong active anchors, use 1-2; if usable only at a broad app/window level, use 3.

Before assigning any 4 or 5, explicitly verify the primary action effect plus at least these active anchors when present: active app/window/title, focused control, exact typed text/path/terminal output, dialog/menu state, selected file/item, visible list/table rows, and window geometry/scroll position.

Discriminative Error Calibration

- Treat the latest action's primary visible effect as a gate. If it is wrong, stale, or missing, set factuality ≤ 2 and quality ≤ 2 even when many background nodes match.
- If the output copies or preserves much of the prior state while the ground truth changes, score it as a stale-state error.
- Downweight generic complete-looking desktop trees: if exact active text/path/cell/menu item, selected/focused element, dialog/menu state, and visible rows/files do not match, total score should normally be ≤ 3 .